

WCF/Web Services Interview Questions & Answers



WCF



By Shailendra Chauhan

Microsoft MVP, Founder & CEO - Dot Net Tricks



WCF/Web Services Interview Questions & Answers

All rights reserved. No part of this book can be reproduced or stored in any retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, uploading on server and scanning without the prior written permission of the Dot Net Tricks Innovation Pvt. Ltd.

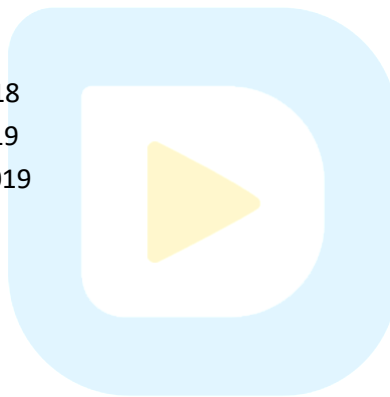
The author of this book has tried their best to ensure the accuracy of the information described in this book. However, the author cannot guarantee the accuracy of the information contained in this book. The author or Dot Net Tricks Innovation Pvt. Ltd. will not be liable for any damages, incidental or consequential caused directly or indirectly by this book.

Further, readers should be aware that the websites or reference links listed in this book may have changed or disappeared between when this book was written and when it is read.

All other trademarks referred to in this book are the property of their respective owners.

Release History

- Initial Release 1.0 - 18th Dec 2018
- Second Release 1.1 - 4th Jan 2019
- Second Release 1.2 - 31st Jan 2019



About Dot Net Tricks

Dot Net Tricks is founded by Shailendra Chauhan (Microsoft MVP), in Jan 2010. Dot Net Tricks came into existence in form of a blog post over various technologies including .NET, C#, SQL Server, ASP.NET, ASP.NET MVC, JavaScript, Angular, Node.js and Visual Studio etc.

The company which is currently registered by a name of Dot Net Tricks Innovation Pvt. Ltd. came into the shape in 2015. Dot Net Tricks website has an average footfall on the tune of 300k+ per month. The site has become a cornerstone when it comes to getting skilled-up on .NET technologies and we want to gain the same level of trust in other technologies. This is what we are striving for.

We have a very large number of trainees who have received training from our platforms and immediately got placement in some of the reputed firms testifying our claims of providing quality training. The website offers you a variety of free study material in form of articles.

Dot Net Tricks Training Solutions

Dot Net Tricks provide you training in traditional as well as new age technologies, via various formats.

Master Courses (Instructor-led)

For a beginner who needs regular guidance, we have a fully packed Master Courses. They are almost equal to semester courses taught in engineering colleges when it comes to length, breadth of content delivery, the only difference instead of 5-6 months, they take approx. 16-weekend classes (2 months).

The detail about Master courses can be found here: <https://www.dotnettricks.com/instructor-led-courses>

Hands-On Learning (Learn to code)

Hands-On Learning courses give you the confidence to code and equally helpful to work in real-life scenarios. This course is composed of hands-on exercise using IDE or cloud labs so that you can practice each and everything by yourself. You can learn to code at your own pace, time and place.

The detail about Hands-On Learning courses can be found here: <https://www.scholarhat.com>

Skill Bootcamps (Instructor-led)

Professionals who don't have two months' time and want to get skilled up in least possible time due to some new project that their company has to take in very near future, we have designed Skill Bootcamps Concept, where you will get trained on consecutive days in a fast-paced manner, where our full focus is going to be on hands-on delivery of technological exercises.

The detail about Skill Bootcamps can be found here: <https://www.dotnettricks.com/skill-bootcamp>

Self-paced Courses

Self-paced courses give you the liberty to study at your own pace, time and place. We understand everyone has their own comfort zone, some of you can afford to dedicate 2 hours a day, some of you not. Keeping this thing in

mind, we created these self-paced courses. While creating these courses we have ensured that quality of courses doesn't get compromise at any parameter, and they also will be able to produce the same results as our other course formats, given the fact you will be able to put your own honest effort.

The detail about Self-paced courses can be found here: <https://www.dotnettricks.com/self-paced-courses>

Corporate Training (Online and Classroom)

Dot Net Tricks having a pool of mentors who help the corporate to enhance their employment skills as per changing technology landscape. Dot Net Tricks offers customized training programs for new hires and experienced employees through online and classroom mode. As a trusted and resourceful training partner, Dot Net Tricks helps the corporate to achieve success with its industry-leading instructional design and customer training initiatives.

The detail about Corporate Training can be found here: <https://www.dotnettricks.com/corporate-training>

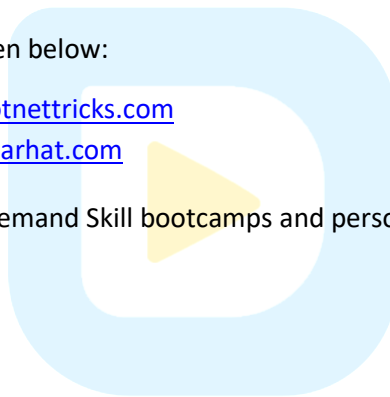
Learning Platform

We have a very robust technology platform to answer the needs of all our trainees, no matter which program they enrolled in. We have a very self-intuitive Learning Management System (LMS), which help you in remain focused and keeping an eye over your progress.

We offer two Learning Platforms as given below:

1. Dot Net Tricks: <https://www.dotnettricks.com>
2. Scholar Hat: <https://www.scholarhat.com>

Apart from these, we also provide on-demand Skill bootcamps and personalized project consultation.



Dedication

My mother Mrs Vriksha Devi and my wife Reshu Chauhan deserve to have their name on the cover as much as I do for all their support made this possible. I would like to say thanks to all my family members Virendra Singh(father), Jaishree and Jyoti(sisters), Saksham and Pranay(sons), friends, to you and to readers or followers of my articles at <https://www.dotnettricks.com/mentors/shailendra-chauhan> to encourage me to write this book.

-Shailendra Chauhan



Introduction

If you are looking for a book on WCF/Web Services interview, you've come across the right book. This is the book which will master you WCF and Web services fundamentals. This book will teach you the WCF from scratch so that you have the confidence to answer the questions asked in your interview.

This book is designed specifically to teach you the SOA architecture concepts along with WCF and ASP.NET Web services features. This book covers SOA concepts, WS-* Standards, WCF features, Binding, message exchange patterns, exception handling, REST principles and security.

So, what where my qualification to write this book? My qualification and inspiration come from my enthusiasm for and the experience with the technology and from my analytic and initiative nature. Being a consultant, corporate trainer, and blogger, I have thorough knowledge and understandings of .NET technologies. My inspiration and knowledge have also come from many years of my working experience and research over it.

So, the next question is who this book is for? This book is best suited for beginners and professionals. It is intended for anyone who is working with .NET technologies like ASP.NET Web Forms, ASP.NET MVC, WPF etc. and would like to implement various messaging strategies. This book covers the mainly following topics.

- SOA Architecture along with Service Oriented Design Principles.
- ASP.NET Web Services core concepts.
- WCF Architecture along with features details.
- Various WCF Contracts and their uses.
- Various WCF Bindings and Message Exchange Patterns.
- WCF Session or Instance Management and Transaction.
- WCF Security implementation like Federated Security, Infra level security etc.
- How to implement WCF versioning and auditing.
- WCF REST core concepts.

This book is not only the WCF/ Web Services interview book but it is more than that. This book helps you to get an in-depth knowledge of WCF and Web Services concepts with a simple and elegant way.

All the best for your interview and happy programming!

About the Author

Shailendra Chauhan - An Entrepreneur, Author, Architect, Corporate Trainer, and Microsoft MVP



He is the **Founder and CEO** of DotNetTricks which is a brand when it comes to e-Learning. DotNetTricks provides training and consultation over an array of technologies like **Cloud, .NET, Angular, React, Node and Mobile Apps development**. He has been awarded as **Microsoft MVP** three times in a row (2016-2018).

He has changed many lives from his writings and unique training programs. He has a number of most sought-after books to his name which have helped job aspirants in **cracking tough interviews** with ease.

Moreover, and to his credit, he has delivered **1000+ training sessions** to professionals worldwide in Microsoft .NET technologies and other technologies including JavaScript, AngularJS, Node.js, React and NoSQL Databases. In addition, he provides **Instructor-led online training, hands-on workshop** and **corporate training** programs.

Shailendra has a strong combination of **technical skills and solution development for complex application architecture with proven leadership and motivational skills** have elevated him to world-renowned status, placing him at the top of the list of most sought-after trainers.

"I always keep up with new technologies and learning new skills to deliver the best to my students," says Shailendra Chauhan, he goes on to acknowledge that the betterment of his followers and enabling his students to realize their goals are his prime objective and a great source of motivation and satisfaction.

Shailendra Chauhan - **"Follow me and you too will have the key that opens the door to success"**

How to Contact Us

Although the author of this book has tried to make this book as accurate as it possible but if there is something strikes you as odd, or you find an error in the book please drop a line via e-mail.

The e-mail addresses are listed as follows:

- mentor@dotnettricks.com
- info@dotnettricks.com

We are always happy to hear from our readers. Please provide your valuable feedback and comments!

You can follow us on [YouTube](#), [Facebook](#), [Twitter](#), [LinkedIn](#) and [Google Plus](#) or subscribe to [RSS feed](#).



Table of Contents

WCF/Web Services Interview Questions & Answers	1
Release History.....	1
About Dot Net Tricks	2
Dot Net Tricks Training Solutions.....	2
Dedication	4
Introduction	5
About the Author	6
How to Contact Us.....	7
Web Services	13
Q1. What is SOA?	13
Q2. What are Service Oriented Design Principles?	13
Q3. What is SOAP?	13
Q4. What is a SOAP message?	14
Q5. What are WS -* Protocols?.....	14
Q6. Why WS -* Protocols introduced?.....	14
Q7. What are Web Services standards and specifications?	14
Q8. What Binding WCF use to implement WS -* standard?.....	15
Q9. What is UDDI?.....	15
Q10. What are Discovery Files?	15
Q11. What is WSDL?	16
Q12. What are Web Services?.....	16
Q13. Explain Web Services Features?	16
Q14. How to create an ASP.NET Web Service?	16
Q15. What is Web service proxy?	17
Q16. What are the advantages of Web Services?.....	17
Q17. Which type of data can be returned by web services web methods?	17
Q18. Does web service support method overloading?.....	17
Q19. Write a sample code for web service method overloading?	17
Q20. Are web services written only in ASP.NET?.....	18
Q21. How to maintain session in Web Services?	18

Q22.	Does a Web service maintain state?	18
Q23.	What are the differences between WCF ASP.NET Web Service?.....	18
Introducing WCF		19
Q1.	What is WCF?	19
Q2.	Explain the features of WCF?	19
Q3.	Explain evolution history of WCF?.....	20
Q4.	Explain WCF 4.5 features?	21
Q5.	Explain WCF 4.0 features?	21
Q6.	What is WCF ABC?	22
Q7.	Explain WCF Architecture?	23
Q8.	What are WCF endpoints?.....	24
Q9.	How to define multiple endpoints for a WCF service?.....	24
Q10.	What are default Endpoints?.....	24
Q11.	What are standard Endpoints?.....	24
Q12.	How to use standard Endpoints?	25
Q13.	What are different WCF contracts?.....	25
Q14.	What is Service Contract?.....	25
Q15.	What is Operation Contract?.....	26
Q16.	What is Data Contract?.....	26
Q17.	What is Message Contract?	26
Q18.	What is the Fault Contract?.....	27
Q19.	What are the various ways of hosting a WCF service?.....	27
Q20.	Explain each type of WCF hosting way?	27
Binding and MEP		29
Q1.	What is Binding?.....	29
Q2.	What are different types of bindings in WCF?	29
Q3.	Compare WCF binding configurations, protocols, security, session and transaction?	30
Q4.	What are default bindings and default protocol mapping in WCF?.....	30
Q5.	How to choose an appropriate binding?	31
Q6.	What are the different Message Exchange Pattern (MEP) in WCF?	32
Q7.	What is the Request-Reply Message Exchange Pattern (MEP) in WCF?	32

Q8.	What is One-Way Message Exchange Pattern (MEP) in WCF?	33
Q9.	What is Duplex Message Exchange Pattern (MEP) in WCF?	34
Q10.	What is Channel Stack?	35
Q11.	What are various Channels in WCF?	35
Q12.	What are the ways to create a WCF Client?	36
Q13.	What is WCF Proxy?	36
Q14.	How to generate a WCF Proxy?	36
Q15.	What is Channel Factory?	36
Q16.	What are the differences between WCF Proxy and Channel Factory?	36
Instance Management		37
Q1.	What is Instance Management in WCF?	37
Q2.	What are various Instance Management ways in WCF?	37
Q3.	Explain each type of Instance Management in WCF?	37
Q4.	How WCF Session is different from ASP.NET Session?	38
Q5.	What is Concurrency Management in WCF?	38
Q6.	Where Concurrency Management is required in WCF?	39
Q7.	What are various Concurrency Modes in WCF?	39
Q8.	Do explain each Concurrency Modes in WCF?	39
Q9.	How Instancing mode and Concurrency mode are related to each other?	41
Q10.	What is Throttling in WCF?	41
Q11.	What are message encoding and decoding in WCF?	42
Q12.	What are different types of message encoding in WCF?	42
Transaction Management		43
Q1.	What are different types of transaction in WCF?	43
Q2.	How you can configure Distributed Transaction in WCF?	43
Q3.	Which protocol support Transaction in WCF?	43
Q4.	Which WCF Bindings support Transaction?	43
Q5.	What is Transaction Flow or Transaction Propagation?	43
Q6.	What is various Transaction Flow?	44
Q7.	Explain all the Transaction Flow options?	44
Q8.	What is Transaction Scope?	45

Q9.	How to allow or disallow a Transaction in WCF?	45
Q10.	How WCF automatically completes the Transaction?.....	45
Q11.	Can we implement transactions in one-way WCF service calls?.....	46
Security and Versioning		47
Q1.	What is Impersonation?	47
Q2.	What are the different types of Impersonation?	47
Q3.	What are Impersonation Levels?.....	48
Q4.	What are various Impersonation Levels?	48
Q5.	What is Delegation?	48
Q6.	How to configure Delegation?.....	48
Q7.	What are various delegation options in WCF?	49
Q8.	What are the differences between impersonation and delegation?	50
Q9.	What are WCF security features?.....	50
Q10.	What is infrastructure level security in WCF?	50
Q11.	What are the various security modes in WCF?	51
Q12.	What is Federated Security in WCF?	52
Q13.	What is a Security Token Service (STS)?	52
Q14.	What is the difference between Credential and Claim?	53
Q15.	What is the protection level in WCF?.....	53
Q16.	How to set protection level in WCF?	53
Q17.	What are different types of protection level?.....	54
Q18.	What is Authentication? What are authentication options in transport level security?	55
Q19.	What is the authentication option in Message-Level Security?.....	55
Q20.	What is authorization? What is the various authorization approach?	56
Q21.	What is Service Auditing?	57
Q22.	What is service versioning?	57
Q23.	What is the different level of versioning?	57
Q24.	What are various version strategies in WCF?.....	58
WCF REST.....		59
Q1.	What is REST?	59
Q2.	Explain the REST principle?	59

Q3.	What is ROA?	59
Q4.	What binding is used for WCF REST?	59
Q5.	What protocols can be used with WCF REST?	60
Q6.	What is the difference between REST and SOAP?	60
Q7.	Explain HTTP GET, POST, PUT and DELETE methods?	60
Q8.	What is UriTemplate?	60
Q9.	What is WCF RIA service?	61
Q10.	What is WCF Data Service?	61
References		62



Web Services

Q1. What is SOA?

Ans. SOA stands for Service Oriented Architecture. It is an architectural design pattern which states that every component of a system should be a service (here service means a unit of a program that serves a business process), and the system should be composed of several loosely-coupled services means services should be independent of each other, so that changing one of them should not affect any other services.

Q2. What are Service Oriented Design Principles?

Ans. There are four Service-oriented principles (tenets) in SOA as given below:

1. **Boundaries are explicit** - This SOA principle states that a service can be deployed anywhere and be easily and freely accessed by other services, regardless of the environment or development language of the other services.
2. **Services are autonomous** - This SOA principle states that each service must be managed and versioned differently so that they do not affect other services in the process. Contracts, once published, should not be changed. Services need to be isolated and decoupled to accomplish the goal of making them autonomous.
3. **Services share schema and contract, not class** - This SOA principle states that services should not pass classes and types; they pass schemas (data) and contracts (behaviours). This allows for a loosely coupled system where the service does not care what type of environment the other service is executing on. The information being passed is 100% platform independent.
4. **Service compatibility is determined based on policy** - This SOA principle states that each service should have its own compatibility level and knows how it would interact with other services. Services look at each other's policy, looking for similarities so that they can start communicating. If two services can't satisfy each other's policy requirements, they are not compatible with each other.

Q3. What is SOAP?

Ans. SOAP stands for Simple Object Access Protocol which is used for exchanging data in XML-based format over HTTP. SOAP is widely-used by web services for exchanging data.

Q4. What is a SOAP message?

Ans. A simple SOAP message structure is given below:

```
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
<soap:Body xmlns:m="http://www.mysite.com/books">
<m:Book>
<m:BookName>WCF and Web Services Interview Questions and Answers</m:BookName>
</m:Book>
</soap:Body>
</soap:Envelope>
```

A SOAP message has an optional header and a required body element. A SOAP message header can contain the application-specific information like authentication or authorization etc. Also, a SOAP message header must be the first child element of the SOAP Envelope element. The SOAP body element contains the actual SOAP message.

Moreover, SOAP is a combination of HTTP protocol and XML i.e. SOAP = HTTP + XML

Q5. What are WS -* Protocols?

Ans. WS -* protocols are a set of protocols (means standards and specifications) that helps us to implement certain needs and behaviours of a service. These protocols describe how to exchange messages in a secure, transactional, and reliable way by using the headers in the SOAP message.

Q6. Why WS -* Protocols introduced?

Ans. SOAP protocol is used for message exchange between Web Services. But SOAP defines very basic semantics for message headers. It does not always define meaning to the header. Hence to extend SOAP, WS-* protocols specifications have been developed with semantics which can then be reused in various application scenarios.

Q7. What are Web Services standards and specifications?

OR

What are WS-* standards and specifications?

Ans. Web service standards and specifications are known as "WS-*" standards. These represent a set of protocols and specifications. These are used to extend the capability of web service.

These standards are defined by big leaders in the industry such as Microsoft, IBM, VeriSign, SUN and many others so that they can expose their services through a common protocol. These standards include:

1. **WS-I Profiles** - The Web Services Interoperability Organization (WS-I) is an industry group to promote interoperability across the stack of web services specifications. It publishes web service profiles, sample applications, and test tools for help. One of the popular profiles it has published is the WS-I Basic Profile. WS-I is governed by a Board of Directors, and Microsoft is one of the board members.
2. **WS-Addressing** - This enables a web service to communicate addressing information. With this new standard, addressing information will be included in the XML message itself. A SOAP header can be added

to the message for this purpose. The network-level transport is now responsible only for delivering that message.

3. **WS-Security** - This describes how to handle security issues within SOAP messages. It attaches signature and encryption information as well as security tokens to SOAP messages. In addition to the traditional HTTP/HTTPS authentications, it incorporates extra security features in the header of the SOAP message, working in the application layer. Also, it ensures end-to-end security.

There are several specifications associated with WS-Security, such as WS-SecureConversation, WS-Federation, WS-Authorization, WS-Policy, WS-Trust, and WS-Privacy.

4. **WS-Reliable Messaging** – This standard enforces reliability between the message source and destination. If a message cannot be delivered to the destination, it must raise an exception or indicate to the source that the message can't be delivered.

There are several delivery assurance options for WS-ReliableMessaging, including `AtLeastOnce`, `AtMostOnce`, `ExactlyOnce`, and `InOrder`.

5. **WS-Coordination and WS-Transaction** - As per OASIS “WS-Coordination specification describe an extensible framework for providing protocols that coordinate the actions of distributed applications. The framework enables existing transaction processing, workflow, and other systems for coordination, to hide their proprietary protocols and to operate in a heterogeneous environment.”

WS-Transaction describes coordination types that are used with the extensible coordination framework described in the WS-Coordination specification.

Q8. What Binding WCF use to implement WS -* standard?

Ans. WCF can implement the WS -* protocols by using **WsHttpBinding**. This binding makes use of some of the WS -* protocols and adds the needed behaviours, such as transactional message calls, reliability, discovery, and addressing.

Q9. What is UDDI?

Ans. UDDI stands for Universal Description, Discovery and Integration. It is a directory service where companies can register their services and search for Web services. UDDI is a public registry, where one can publish and inquire about web services. UDDI communicates via SOAP protocol and uses WSDL to describe interfaces to web services.

Q10. What are Discovery Files?

Ans. The Discovery files are XML based files which are used to group common services together on a web server. The Discovery files contain a link in the form of URLs to resources that provide discovery information for a web service.

Q11. What is WSDL?

Ans. WSDL stands for Web Services Description Language. It is an XML format that contains all the metadata about the web service including function names, parameter names, and their types and the types of return Values.

The purpose of WSDL is to describe Web services and how to access them. It can be used by any development environments to create classes in their programming language (such as J2EE).

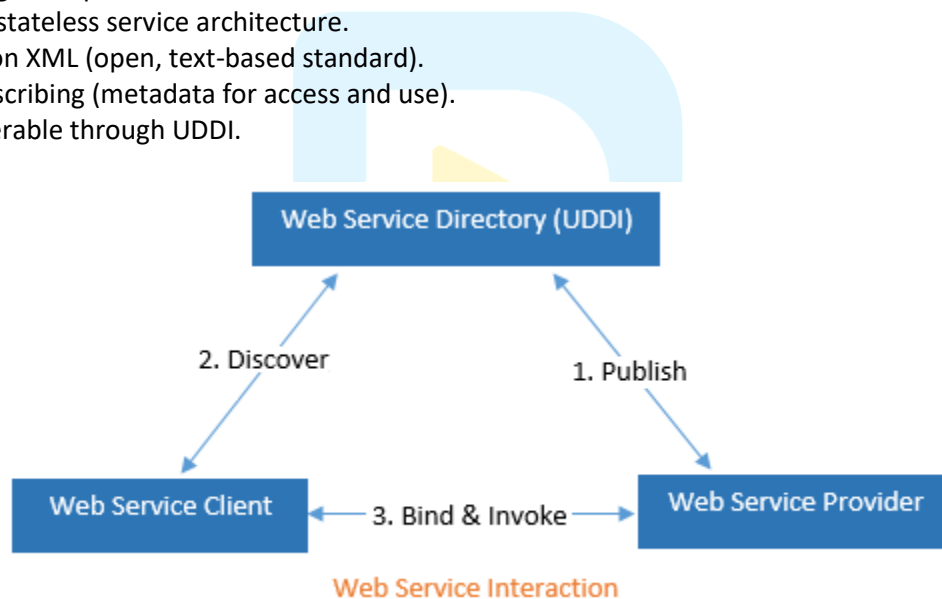
Q12. What are Web Services?

Ans. A Web service is a set of operations exposed over the internet and consumed by various clients. A web service is typically hosted on a server and called by a client application (consumer) over a network. Web Service uses HTTP protocol and message in XML format for communications.

Q13. Explain Web Services Features?

Ans. Following are the web services key features:

1. Platform Independent.
2. Language Independent.
3. Follow stateless service architecture.
4. Based on XML (open, text-based standard).
5. Self-describing (metadata for access and use).
6. Discoverable through UDDI.



Q14. How to create an ASP.NET Web Service?

Ans. An ASP.NET Web service can be created by adding an ASMX file using Visual Studio. A code example for Web Service is given below:

```
<%@ WebService Language="C#" Class="Utility" %>
using System.Web.Services;
using System;
public class Utility: WebService
{
    [WebMethod]
    public long Multiply(int a, int b)
    {
```

```

        return a * b;
    }
}

```

Q15. What is Web service proxy?

Ans. A web service proxy is a stub class between a web service and a client which contains stub methods representing all of the methods of the web service. A client application can call web service's method by using these stub methods. It also contains other necessary information required by the client to call the web service such as custom exceptions, customs data and class types, and so on.

A web service proxy is generally auto-generated by Visual Studio IDE or any other tool. A proxy class is always for a specific language means Java clients and C# clients would have their own proxy classes.

Q16. What are the advantages of Web Services?

Ans. There are following main advantages of using Web Services -

1. Web services are platform and language independent.
2. Web Services make the communication easy between the applications.
3. Easy to reuse existing Web services.
4. Works in the Heterogeneous environment.
5. Rapid development.

Q17. Which type of data can be returned by web services web methods?

Ans. Web method can return all the primitive data types and custom data types which can be encoded and serialized by XML.

Q18. Does web service support method overloading?

Ans. By default, a Web service does not support method overloading. But method overloading can be achieved by setting *MessageName* property of *WebMethod* attribute and commenting the *WebServiceBinding* attribute at the service level.

Q19. Write a sample code for web service method overloading?

Ans. The sample code for the webservice method overloading is given below:

```

[WebService(Namespace = "http://tempuri.org/")]
// [WebServiceBinding]
[System.ComponentModel.ToolboxItem(false)]
public class WebService1: System.Web.Services.WebService
{
    [WebMethod(MessageName = "HelloWorld1")]
    public string HelloWorld()
    {
        return "Hello World";
    }
    [WebMethod(MessageName = "HelloWorld2")]
    public string HelloWorld(string msg)
    {
        return "Hello World" + msg;
    }
}

```

```
}  
}
```

Q20. Are web services written only in ASP.NET?

Ans. No, web services can also be written by other languages also like Java, PHP etc.

Q21. How to maintain session in Web Services?

Ans. You can maintain session in a web service method by setting the *WebMethod* attribute's *EnableSession* property value *true*.

Q22. Does a Web service maintain state?

Ans. Web services are based on HTTP protocol which is stateless. Hence, by default ASP.NET Web services do not have any technique to maintain state. However, it can access ASP.NET objects, such as application and session.

Also, you can maintain session in a web service method by setting the *WebMethod* attribute's *EnableSession* property value *true*.

Q23. What are the differences between WCF ASP.NET Web Service?

Ans. The differences between ASP.NET Web Service and WCF are given below:

WCF	ASP.NET Web Service (ASMX Service)
ServiceContract and OperationContract attributes are used for defining WCF service.	WebService and WebMethod attributes are used for defining web service.
Supports various protocols like HTTP, HTTPS, TCP, Named Pipes and MSMQ.	Supports only HTTP, HTTPS protocols.
Hosted in IIS, WAS (Windows Activation Service), Self-hosting, Windows Service.	Hosted only in IIS.
Supports security, reliable messaging, transaction and AJAX and REST supports.	Support security but is less secure as compared to WCF.
Supports DataContract serializer by using System.Runtime.Serialization.	Supports XML serializer by using System.Xml.Serialization.
Supports One-Way, Request-Response and Duplex service operations.	Supports One-Way and Request-Response service operations.
WCF is faster than Web Services.	Web Services are slower than WCF
HashTable can be serialized.	HashTable cannot be serialized. It can serialize only those collections which implement IEnumerable and ICollection.
Unhandled Exceptions does not return to the client as SOAP faults. WCF supports better exception handling by using FaultContract.	Unhandled Exceptions returns to the client as SOAP faults.
Supports XML, MTOM, Binary message encoding.	Supports XML and MTOM (Message Transmission Optimization Mechanism) message encoding.
Supports multi-threading by using <i>ServiceBehaviour</i> class.	Doesn't support multi-threading.

2

Introducing WCF

Q1. What is WCF?

Ans. WCF is a programming model for building and developing the service-oriented application. WCF allows applications to communicate with each other in the distributed environment. WCF is a set of technologies that covers **ASMX** web services, Web Services Enhancements (**WSE**), .NET **Remoting** and **MSMQ**.

The purpose of WCF is to provide a single programming model that can be used to create services on the .NET platform for organizations.

Q2. Explain the features of WCF?

Ans. WCF includes the following set of features:

- **Service Orientation** - WCF allows us to create *service-oriented* applications means applications use WCF to send and receive the data.
- **Interoperability** – Interoperability means the ability to exchange and use information within a heterogeneous network/platform. WCF follows modern industry standards for Web service interoperability.
- **Multiple Message Patterns** – WCF supports multiple message exchange pattern such as One Way Message Exchange Pattern (In this client will send the message and does not bother about server response), Request/Reply Message Exchange Pattern (In this client sends the message and server receives the message and will send response to client.) and Duplex Message Exchange Pattern (In this Sender and Receiver can exchange multiple messages means Sender can become Receiver and Receiver can become the Sender).
- **Service Metadata** – WCF can publish the service metadata in various standards formats such as WSDL, XML Schema and WS-Policy. This metadata is used to generate and configure clients for accessing a WCF service.
- **Security** – In WCF, Messages can be encrypted to make it secure and you can also force users to authenticate themselves before receiving messages.

- **Multiple Transports and Encodings** – In WCF, Messages can be sent by using multiple transport protocols (HTTP, HTTPS, TCP, and named pipes or MSMQ) and encodings (binary, text, and Message Transmission Optimization Mechanism (MTOM)).
- **Reliable and Queued Messages** - WCF supports reliable message exchange i.e. it provides message transfer assurances from a source to a destination irrespective of message transfer failure. This is achieved through reliable sessions implementation over WS-Reliable Messaging and using MSMQ.
- **Durable Messages** - A durable message is that which never lost due to disruption in the communication. Typically, messages in a durable message pattern are stored to a database and in case of disruption, the database will help you to resume the message exchange when the connection is restored.
- **Transactions** - In WCF, a transaction can be supported by using one of the transaction models: WS-Atomic transactions, the APIs in the *System.Transactions* namespace, and Microsoft Distributed Transaction Coordinator.
- **AJAX and REST Support** - WCF can also be extended to support various XML formats like as ATOM (a popular RSS standard). It can also be used to support non-XML formats like as JSON. You can also configure WCF to support REST.

Q3. Explain evolution history of WCF?

Ans. The WCF evolution history is given below:

WCF Version	Introduced with IDE	Features Detail
4.5	Visual Studio 2012	• Task-based Async Support • Contract-First Development • WCF Configuration Validation • Web Socket Support • UDP Endpoint Support • New Https protocol mapping on IIS • Streaming Improvements • Multiple Auth support for single endpoint
4.0	Visual Studio 2010	• Simple Configuration • Serialization Enhancements • Support for WS-Discovery • Routing Service • Standard Endpoints • Workflow Service

3.5	Visual Studio 2008	• UriTemplates Support • Support for REST Style Services • ASP.NET Ajax Integration and JSON support • Added WS* Specification Support • Support for RSS and Atom feeds
3.0	Visual Studio 2005	• Introduced the first version of WCF with many features like Address, Binding, Contract, Sessions, Instancing, and Concurrency management

Q4. Explain WCF 4.5 features?

Ans. WCF 4.5 features details are given below -

1. **Task-based Async Support** using the new Task-based async programming model.
2. **Simplified Generated Configuration Files** - In WCF 4.5 the generated configuration files contain only those binding properties that are set to a non-default value.
3. **Contract-First Development** - WCF also supports contract-first development since the svcutil.exe has a /serviceContract switch which can generate service and data contracts from a WSDL document.
4. **ASP.NET Compatibility Mode Default Changed**- By default `AspNetCompatibilityRequirements` Attribute is now set to Allowed.
5. **Streaming Improvements** – Now WCF has support for asynchronous streaming where the sender side does not block threads if the receive side is not reading or slow in reading. In this way it is increasing the scalability by removing the limitation of message buffering when a client sends a streamed message to an IIS hosted WCF service.
6. **WebSockets Support** - Two new bindings `NetHttpBinding` and `NetHttpsBinding` have been added to support communication over a WebSocket transport.
7. **Multiple Authentication Support**- WCF supports multiple authentication modes provided by IIS when you use the HTTP transport and transport security.

Q5. Explain WCF 4.0 features?

Ans. WCF 4.0 features details are given below -

1. **Configuration Based Activation**- While hosting a Windows Communication Foundation (WCF) service under Internet Information Services (IIS) or Windows Process Activation Service (WAS), you must provide a .svc file. The configuration-based activation feature of WCF eliminates the need to have a .svc file and the related overhead.

2. **Routing Services** - A Routing Service acts as a message router which allows a message to be sent to a client based on a value within the message header or message body.
3. **Multiple IIS Site Bindings Support**- WCF has multiple IIS Site Bindings Support. It is useful to create a service that has a base address for internal users and a separate base address for external users.
4. **Support for WS-Discovery** - The Service Discovery feature enables client applications to dynamically discover service addresses at runtime in an interoperable way using WS-Discovery.
5. **Supports for HTTP Decompression**- WCF clients now have built-in support for HTTP decompression. A client can decompress received messages that were compressed with the gzip/deflate format.
6. **Standard Endpoints**- Standard endpoints are pre-defined endpoints that have one or more of their properties (address, binding, and contract) fixed.
7. **Target Framework Attribute**- The target framework attribute is used to specify the version of the .NET Framework an application hosted in IIS or WAS is targeting. It allows you to build applications that target .NET Framework 2.0, 3.5, or 4 using Visual Studio.

Q6. What is WCF ABC?

Ans. WCF ABC stands for Address, Binding and Contract.

1. **Address (Where)** – WCF Address specifies to a specific location, where the service is hosted. A WCF Address is specified as a URI. URI's first part specifies the transport protocol (HTTP, TCP, Net.pipe and MSMQ), and the others parts specify the host machine address and service path as shown in below fig.



Others sample Addresses are given below:

```
http://www.mywebsite/MyService
net.tcp://www.mywebsite/MyService
net.pipe://www.mywebsite/MyPipeService
net.msmq://www.mywebsite/MyMsmqService
```

2. **Binding (How)** – WCF binding specifies how the service and client will communicate with each other in terms of transport protocols and encoding (such as text or binary).

WCF binding is a set of binding elements and each element specify, how the service and client will communicate with each other. Each binding must have at least one transport element and one message encoding element.

3. **Contract (What)** - WCF contract specifies what the service contains. WCF has five types of contracts: service contract, operation contract, data contract, message contract and fault contract. Each contract defines a certain behaviour.

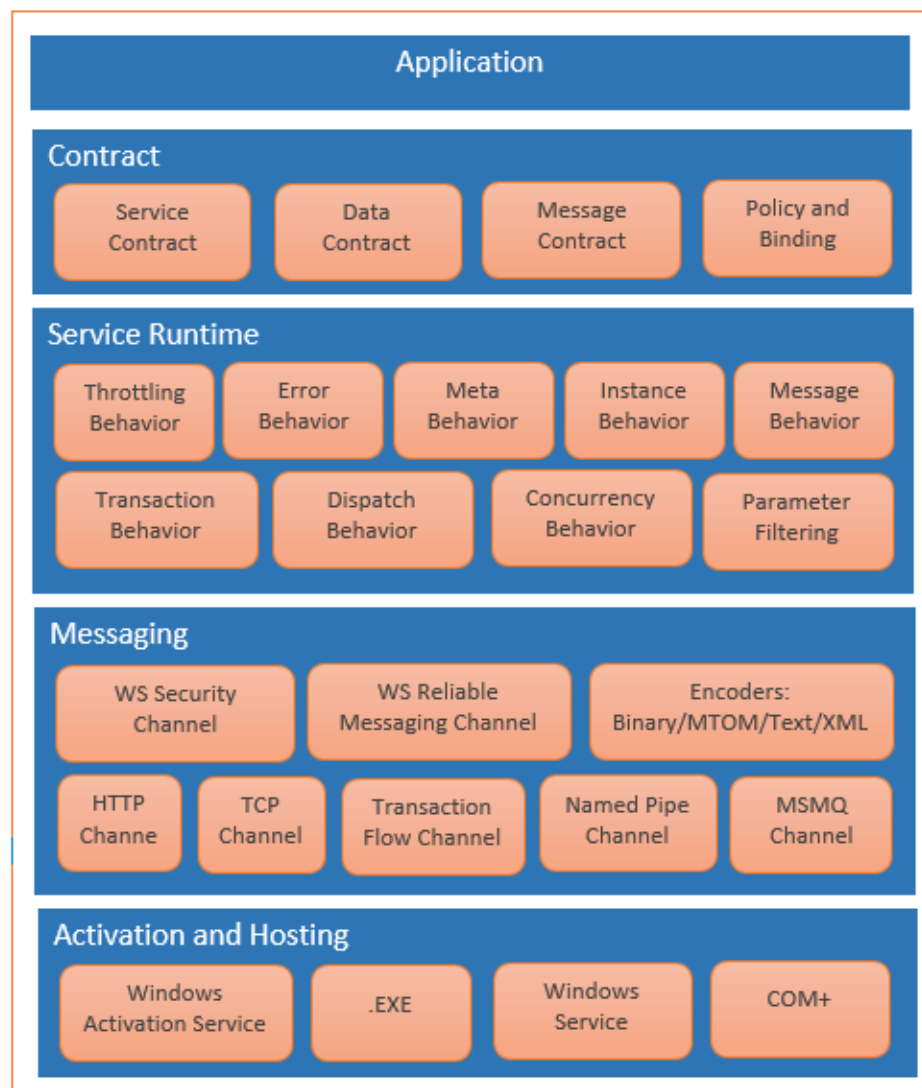
Q7. Explain WCF Architecture?

Ans. WCF has a layered architecture that can be divided into following layers which are described as:

Application Layer – This layer describes how the application consumes the WCF service.

Contract Layer – This layer describes what a service does and what will be the Endpoints for communication to the outside world.

Service Runtime Layer - This layer specifies and manages the behaviours of the service that occur during service operation.

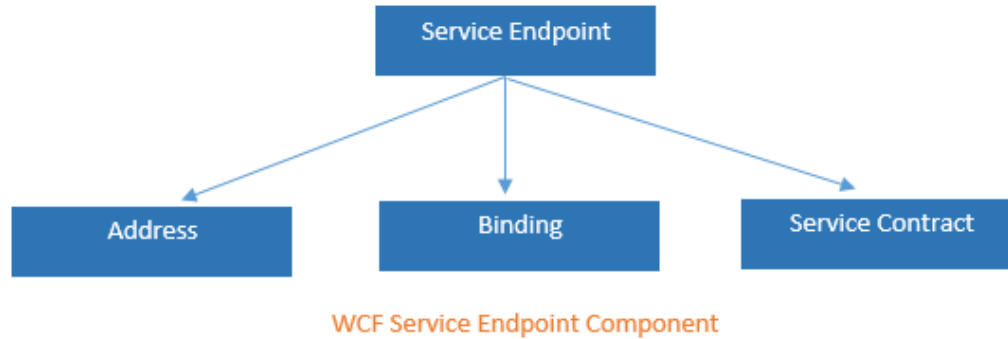


Messaging - This layer defines what format and data exchange patterns can be used during service communication.

Activation and Hosting - This layer provides various options in which a service can be started as well as hosted.

Q8. What are WCF endpoints?

Ans. WCF endpoints are used to expose and consume a WCF service. An endpoint includes all the information which is required for the message exchange. A service can expose one or more endpoints. A WCF service endpoint has an address, a binding, and a service contract.



Q9. How to define multiple endpoints for a WCF service?

Ans. A WCF service configuration with multiple endpoints is given below:

```
<system.serviceModel>
<services >
  <service name = "MyWCFService">
    <endpoint address = "http://localhost:90/MyWCFService"
      binding = "wsHttpBinding" contract = "IMyWCFContract"/>
    <endpoint address = "net.tcp://localhost:91/MyWCFService"
      binding = "netTcpBinding" contract = "IMyWCFContract"/>
  </service>
</services>
</system.serviceModel>
```

Q10. What are default Endpoints?

Ans. WCF offers some default endpoints to the service if a service host doesn't define any endpoints but has at least one base address. **For example**, WCF uses the basic binding for the Http address.

Q11. What are standard Endpoints?

Ans. WCF provides a set of pre-defined endpoints known as Standard Endpoints for metadata exchange, discovery and web. You can configure the standard endpoints by using a config file and programmatically. Here is the list of standard endpoints:

- mexEndpoint
- webHttpEndpoint
- webScriptEndpoint

- workflowControlEndpoint
- announcementEndpoint
- discoveryEndpoint
- udpAnnouncementEndpoint
- udpDiscoveryEndpoint

Q12. How to use standard Endpoints?

Ans. To use the above-defined standard points, you have to use the kind attribute as given below:

```
<system.serviceModel>
<services>
  <service name="MyWCFService">
    <host>
      <baseAddresses>
        <add baseAddress = "http://localhost:90/MyWCFService"/>
        <add baseAddress = "net.tcp://localhost:91/MyWCFService"/>
      </baseAddresses>
    </host>
    <endpoint kind="mexEndpoint" />
  </service>
</services>
</system.serviceModel>
```

Q13. What are different WCF contracts?

Ans. WCF contract specifies the service and its operations. WCF has five types of contracts:

1. Service contract
2. Operation contract
3. Data contract
4. Message contract
5. Fault contract.

Q14. What is Service Contract?

Ans. A service contract defines the operations which are exposed by the service to the outside world. A service contract is the interface of the WCF service and it tells the outside world what the service can do. It may have service-level settings, such as the name of the service and namespace for the service.

```
[ServiceContract]
interface IMyContract
{
    [OperationContract]
    string MyMethod();
}
class MyService: IMyContract
{
    public string MyMethod()
    {
        return "Hello World";
    }
}
```

```
}
```

Q15. What is Operation Contract?

Ans. An operation contract is defined within a service contract. It defines the parameters and returns type of operation. An operation contract can also define operation-level settings, like as the transaction flow of the operation, the directions of the operation (one-way, two-way, or both ways), and fault contract of the operation.

```
[ServiceContract]
interface IMyContract
{
    [FaultContract(typeof(MyFaultContract))]
    [OperationContract]
    string MyMethod();
}
```

Q16. What is Data Contract?

Ans. A data contract defines the data type of the information that will be exchanged between the client and the service. A data contract can be used by an operation contract as a parameter or return type, or it can be used by a message contract to define elements.

```
[DataContract]
class Person
{
    [DataMember]
    public string ID;
    [DataMember]
    public string Name;
}
[ServiceContract]
interface IMyContract
{
    [OperationContract]
    Person GetPerson(int ID);
}
```

Q17. What is Message Contract?

Ans. When an operation contract required passing a message as a parameter or returning value as a message, the type of this message will be defined as a message contract. A message contract defines the elements of the message (like as Message Header, Message Body), as well as the message-related settings, such as the level of message security.

Message contracts give you complete control over the content of the SOAP header and the SOAP body.

```
[ServiceContract]
public interface IRentalService
{
    [OperationContract]
    double CalPrice(PriceCalculate request);
}
[MessageContract]
public class PriceCalculate
{
}
```

```

    [MessageHeader]
    public MyHeader SoapHeader { get; set; }
    [MessageBodyMember]
    public PriceCal PriceCalculation { get; set; }
}
[DataContract]
public class MyHeader
{
    [DataMember]
    public string UserID { get; set; }
}
[DataContract]
public class PriceCal
{
    [DataMember]
    public DateTime PickupDateTime { get; set; }
    [DataMember]
    public DateTime ReturnDateTime { get; set; }
    [DataMember]
    public string PickupLocation { get; set; }
    [DataMember]
    public string ReturnLocation { get; set; }
}

```

Q18. What is the Fault Contract?

Ans. A fault contract defines errors raised by the service, and how the service handles and propagates errors to its clients. An operation contract can have zero or more fault contracts associated with it.

```

[ServiceContract]
interface IMyContract
{
    [FaultContract(typeof(MyFaultContract1))]
    [FaultContract(typeof(MyFaultContract2))]
    [OperationContract]
    string MyMethod();

    [OperationContract]
    string MyShow();
}

```

Q19. What are the various ways of hosting a WCF service?

Ans. There are four ways of hosting a WCF service.

- Self-Hosting
- Windows services hosting
- IIS hosting
- Windows Activation Services hosting (WAS)

Q20. Explain each type of WCF hosting way?

Ans. Each type of WCF hosting explanation is given below:

1. Self-hosting

WCF supports self-hosting and this behaviour makes the WCF service to be hosted within any kind of .NET application. The self-hosting is the most flexible and easiest way of hosting a WCF service and also supports all bindings and transports. The self-hosted service will be only available when the application is running.

2. Windows services hosting

A WCF service can be hosted as a Windows service. A Windows service is a process managed by the OS and it is automatically started when Windows OS is started (if it is configured to do so). This hosting doesn't support some features like as versioning for WCF services.

3. IIS hosting

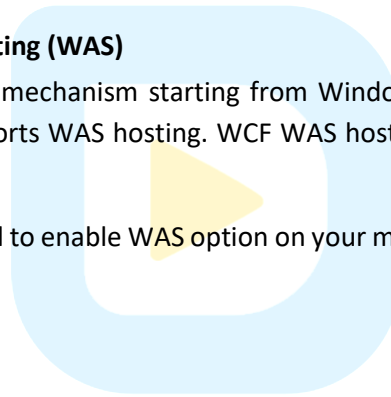
A WCF service can be hosted on IIS and can take benefits of many useful features of IIS, such as process recycling, idle shutdown, process health monitoring, high availability, easy manageability, versioning, and easy deployment scenarios.

All of these features are required for enterprise-level WCF services. Windows XP and Windows Server 2003 have IIS6 which has only support for WCF hosting using the HTTP protocol.

4. Windows Activation Services hosting (WAS)

WAS is a new process activation mechanism starting from Windows Server 2008 and Windows Vista for hosting WCF Service. IIS 7.0 supports WAS hosting. WCF WAS hosting supports non-HTTP protocols like as TCP, named pipes and MSMQ.

For WAS hosting of WCF, you need to enable WAS option on your machine using "Turn Windows features on or off".



Binding and MEP

Q1. What is Binding?

Ans. WCF binding specifies how the service and client will communicate with each other in terms of transport protocols and encoding (such as text or binary).

WCF binding is a set of binding elements and each element specifies, how the service and client will communicate with each other. Each binding must have at least one transport element and one message encoding element.

Q2. What are different types of bindings in WCF?

Ans. WCF supports the following types of built-in bindings:

1. **Basic binding** - This binding is supported by BasicHttpBinding class and designed to expose a WCF service as an ASMX web service. The benefit of this binding is if there are old clients (who are still using ASMX web service) can consume new service.
2. **Web binding** – This binding is supported by the WebHttpBinding class and designed to expose a WCF service over HTTP protocol. Typically, it is used to develop REST-based service using WCF.
3. **Web Service (WS) binding** – This binding is supported by the WSHttpBinding class. Typically, it is a combination of Basic binding and WS-* specifications which are not supported by Basic binding.

```
wsHttpBinding = basicHttpBinding + WS-* specifications
```

4. **WS Dual binding** – This binding is supported by the WsDualHttpBinding class. Typically, it is a combination of wsHttpBinding and bi-directional communication. In this binding, both clients and services can send and receive messages to each other.
5. **TCP binding** – This binding is supported by the NetTcpBinding class. This binding uses TCP protocol and best suited for the intranet-based applications and faster than Http protocol-based bindings.
6. **IPC binding** - This binding is supported by the NetNamedPipeBinding class. This binding uses named pipe for the communication and best suited for the communication between services hosted on the same machine. This is the most secure and fastest binding among all the bindings.
7. **MSMQ binding** - This binding is supported by the NetMsmqBinding class. This binding uses MSMQ for the communication and best suited for the disconnected applications. It offers a disconnected messages queue for the communication between client and server.

8. **Federated WS binding** - This binding is supported by the WSFederationHttpBinding class. This binding includes WS binding with federated security.
9. **Peer Network binding** - This binding is supported by the NetPeerTcpBinding class. This binding uses the TCP protocol along with peer networking (In this each machine i.e. node acts as a client or a server to the other nodes).
10. **MSMQ integration binding** - This binding is supported by the MsmqIntegrationBinding class and uses MSMQ for the communication with the existing systems.

Q3. Compare WCF binding configurations, protocols, security, session and transaction?

Ans. Comparison of the above things is given below-

Binding	Configuration	Protocol/ Transport	Default Security	Default Session	Transaction	Duplex	Default Encoding
BasicHttpBinding	Basic Profile 1.1	Http, Https	None	No	No	No	Text
WSHttpBinding	WS	Http, Https, TCP	Message	Optional	Yes	No	Text
WSDualHttpBinding	WS	Http, Https	Message	Yes	Yes	Yes	Text
NetTcpBinding	.NET	TCP	Transport	Optional	Yes	Yes	Binary
NetNamedPipeBinding	.NET	Named Pipe	Transport	Yes	Yes	Yes	Binary
NetMsmqBinding	.NET	MSMQ	Transport	Yes	Yes	No	Binary
WSFederationHttpBinding	WS-federation	Http, Https	Message	Yes	Yes	No	Text
NetPeerTcpBinding	Peer	TCP	Transport	No	No	Yes	-
MsmqIntegrationBinding	MSMQ	MSMQ	Transport	Yes	Yes	No	N/A

Q4. What are default bindings and default protocol mapping in WCF?

Ans. WCF offers some default bindings which enable us to create a service that will be automatically set to the correct binding by specifying the address of your service (or the base address). This is achieved by using protocol mapping as shown given below:

WCF 4.0 - Default Bindings:

Schema	WCF Binding
Http	BasicHttpBinding
Net.tcp	NetTcpBinding
Net.msmq	NetMsmqBinding
Net.pipe	NetNamedPipeBinding

The default binding is also referred to as nameless binding. For TCP protocol the default binding would be as:

```
<netTcpBinding>
  <binding transactionFlow = "true"/>
</netTcpBinding>
```

WCF 4.0 - Default Protocol Mapping

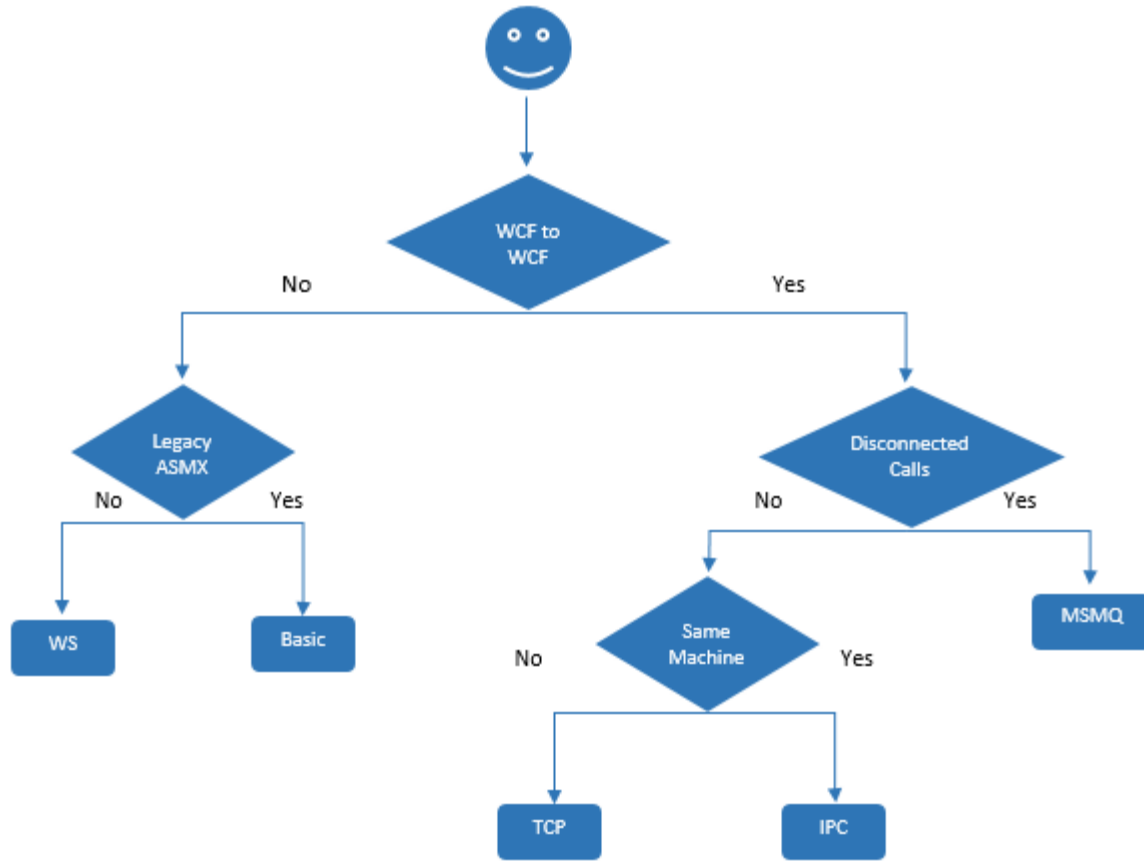
```
<system.serviceModel>
<protocolMapping>
<add scheme="http" binding="basicHttpBinding"/>
<add scheme="net.tcp" binding="netTcpBinding"/>
<add scheme="net.pipe" binding="netNamedPipeBinding"/>
<add scheme="net.msmq" binding="netMsmqBinding"/>
</protocolMapping>
</system.serviceModel>
```

Above default protocol mapping can be changed, or you can also add your own protocol mapping as shown below:

```
<system.serviceModel>
<protocolMapping>
<add scheme="http" binding="wsHttpBinding"/>
<add scheme="https" binding="wsDualHttpBinding"/>
</protocolMapping>
</system.serviceModel>
```

Q5. How to choose an appropriate binding?

Ans. Depending upon your requirements, you can choose a binding for your service as shown in the diagram:



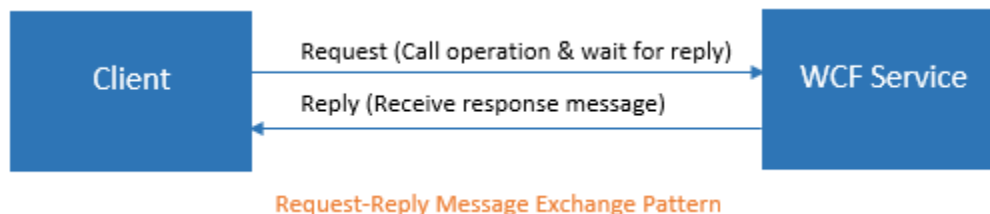
Q6. What are the different Message Exchange Pattern (MEP) in WCF?

Ans. Message Exchange Patterns describes the way of communication between Client and Server means how client and server would be exchange messages to each other. There are three types of message exchange patterns:

1. Request-Reply
2. One-Way
3. Duplex

Q7. What is the Request-Reply Message Exchange Pattern (MEP) in WCF?

Ans. In this communication, the client sends the message to the service and waits for a reply from the service. Within a *ReceiveTimeout* period (default timeout is one minute), if the service doesn't respond to the client then the client will receive a *TimeoutException*. In this pattern, the client waits for the reply message, even if the service operation's return type is *void*.



Request-Reply Message Exchange Pattern

This is the by default message exchange pattern in WCF. All WCF bindings except MSMQ-based bindings support this pattern.

Example:

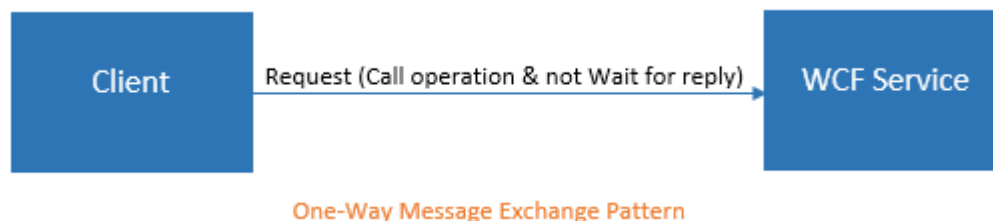
```
[ServiceContract]
public interface RequestReplyService
{
    [OperationContract]
    string GetData(int value);

    [OperationContract(IsOneWay = false)]
    void SaveData(string value);
}
```

To define this pattern, you can set *IsOneWay* property to false explicitly, but by default it is false. So there is a need to define *IsOneWay* property for the Request-Reply pattern. A Request-Reply operation returns header with an HTTP status code of 200 (OK) and a full SOAP response in the message body.

Q8. What is One-Way Message Exchange Pattern (MEP) in WCF?

Ans. In this communication, the client sends the message to the service and doesn't wait for a reply from the service. In this pattern, the receiver doesn't send any response to the sender, even if an error occurs in the communication.



This pattern doesn't support output parameters, by-reference parameters and return value to an operation, otherwise, you will receive an *InvalidOperationException*.

All of the WCF bindings support one-way operations.

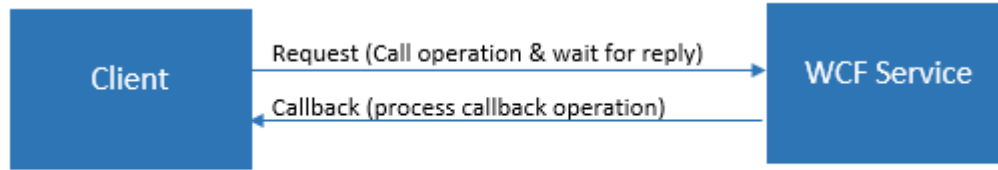
Example:

```
[ServiceContract]
public interface OneWayService
{
    [OperationContract(IsOneWay = true)]
    void SaveData(string value);
}
```

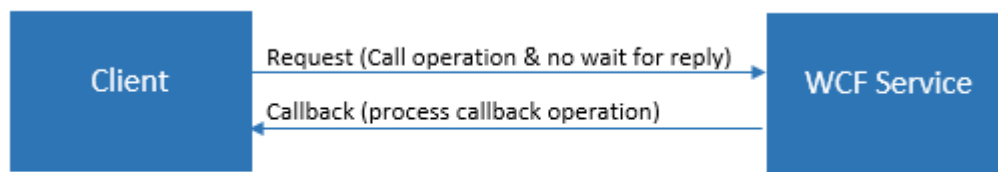
A One-Way operation returns the only header with an HTTP status code of 202 (Accepted) without a message body. This pattern is commonly used with per-call or singleton services only.

Q9. What is Duplex Message Exchange Pattern (MEP) in WCF?

Ans. In this communication, the client and services can send messages to each other by using One-way or request-reply messaging.



Duplex (Request-Reply) Message Exchange Pattern



Duplex (One-Way) Message Exchange Pattern

Only bidirectional-capable bindings support this pattern like as WS Dual, TCP and IPC bindings.

To make a duplex contract, you must also define a callback contract and assign the *typeof* that callback contract to the *CallbackContract* property of your service contract's *ServiceContract* attribute as shown in below example.

```
[ServiceContract(CallbackContract = typeof(DuplexServiceCallback))]  
public interface DuplexService  
{  
    [OperationContract(IsOneWay = true)] //One-Way  
    void SaveData();  
  
    [OperationContract] //Request-Reply.  
    string GetData();  
}  
  
public interface DuplexServiceCallback  
{  
    [OperationContract(IsOneWay = true)]  
    void Progress(string status);  
}
```

For this pattern, you must also specify the binding as *wsDualHttpBinding* within your web.config as shown below:

```
<services>  
  <service name="WCFServiceApp.DuplexService">  
    <endpoint address="" binding="wsDualHttpBinding"  
contract="WCFServiceApp.IDuplexService">  
    </endpoint>  
  </service>  
</services>
```

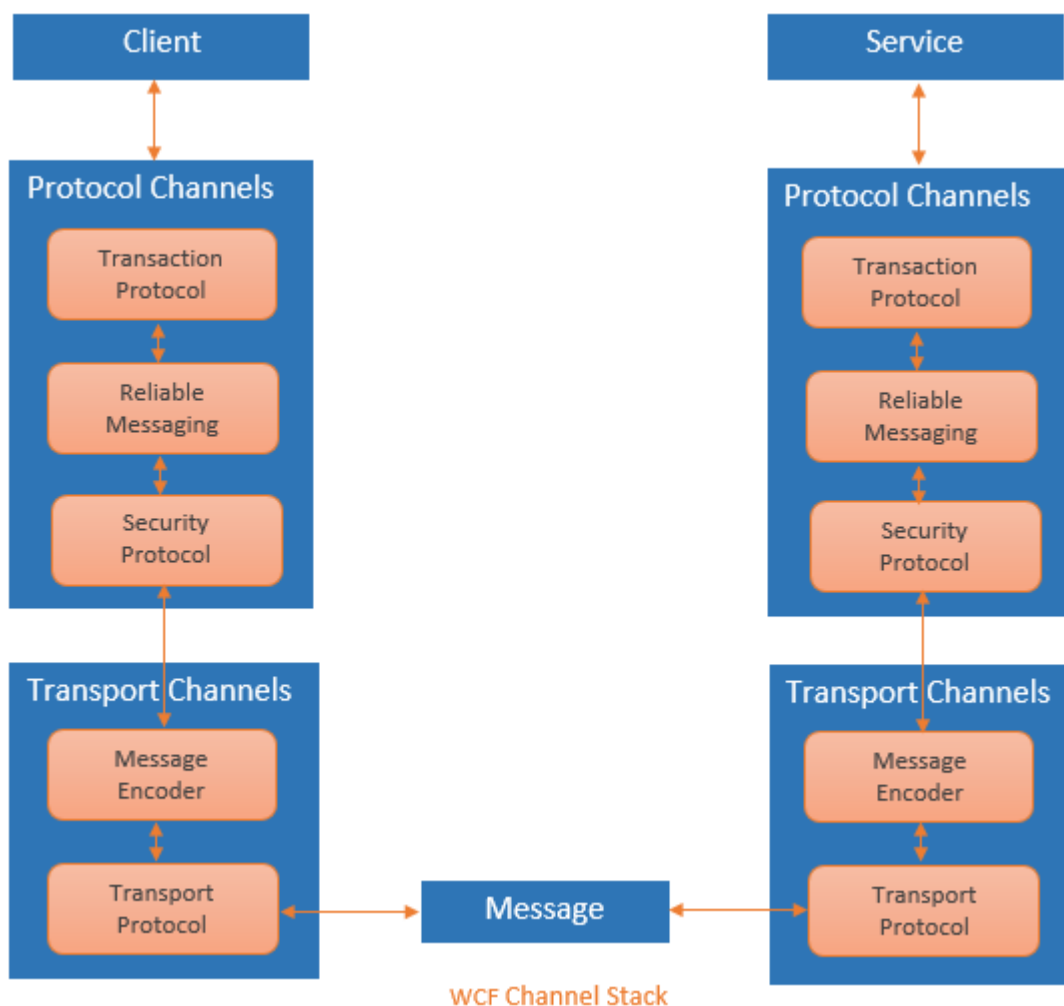
Q10. What is Channel Stack?

Ans. A WCF channel stack is a layered stack including at least one or more channels which are used for the communication. Each binding element in the WCF binding is corresponding to a specific channel in the WCF Channel Stack.

Q11. What are various Channels in WCF?

Ans. There are following two types of channels in WCF:

Transport Channels - These are responsible for message encoding (like text, binary or MTOM) and sending or receiving the message using a transport protocol like HTTP, TCP, MSMQ, Peer to Peer and Named Piped. Also, transport channels always exist at the bottom of the channels stack.



Protocol Channels - These are responsible for implementing security, transaction and reliable messaging.

At sender side, *ChannelFactory* is used to build a channel stack for sending the messages. At the receiver side, *ChannelListener* is used to build a channel stack for listening to the incoming messages.

Q12. What are the ways to create a WCF Client?

Ans. There are two ways to create a WCF Client or calling a WCF Service as:

- WCF Proxy
- Channel factory

Q13. What is WCF Proxy?

Ans. A WCF proxy is a CLR class that exposes the service contract. A service proxy includes the service contract operations and other methods for managing the proxy life cycle.

Q14. How to generate a WCF Proxy?

Ans. A WCF proxy can be generated with the help of Visual Studio by adding a service reference to the application. You can also generate a WCF proxy by using *SvcUtil.exe* command-line utility.

Q15. What is Channel Factory?

Ans. A channel factory creates channels of different types that are used by clients to send messages to the service. We use *ChannelFactory* class with a known interface to create the channel. This approach is commonly used when you have access control to both the server and the client.

```
var channelFactory = new ChannelFactory<IService>("WSHttpBinding_IService");  
IService channel = channelFactory.CreateChannel();  
channel.MyOperation();  
((IClientChannel)channel).Close();
```

Q16. What are the differences between WCF Proxy and Channel Factory?

Ans. The differences between WCF Proxy and Channel Factory is given below -

Proxy	Channel Factory
Only requires the service URL to access the service.	Requires direct access to the assembly which contains the service contract means need to add service DLL into the application to access the service.
Simple and easy to understand	Not easy since channels are complex and network-related
Proxy is best when your service entities will change frequently or used by several applications.	Channel Factory is best when your service entities will not change frequently or tightly bound to the single application.
Create a proxy by using SVCUtil.exe	Channel factory class is used to create a channel and for accessing the service.

4

Instance Management

Q1. What is Instance Management in WCF?

OR

How WCF Service manage Session?

Ans. WCF manage session by creating the instance of the service class. This created instance(s) handle the incoming service request. In WCF, the session is the way of managing the services instance(s) so that server can use these instances in an optimized way. At the server side, the *InstanceContext* class is used to manage service class instance.

Q2. What are various Instance Management ways in WCF?

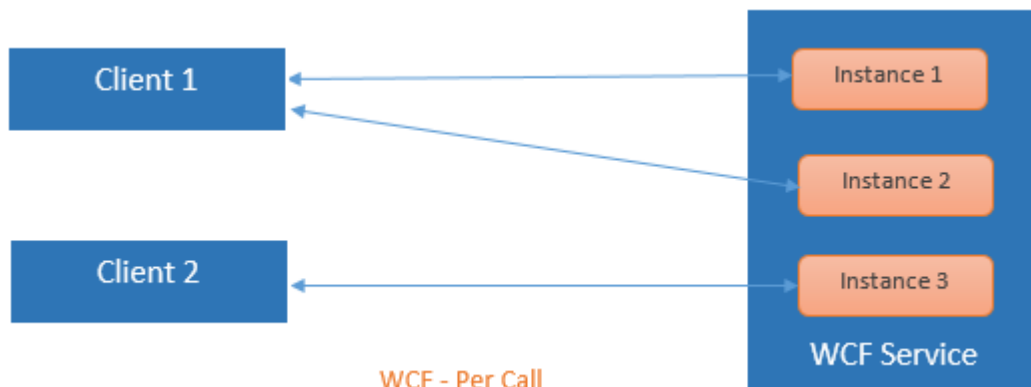
Ans. There are three ways to manage instance in WCF:

1. Per Call
2. Per Session
3. Single

Q3. Explain each type of Instance Management in WCF?

Ans. The explanation about each instance management is given below:

1. **Per Call** – In this way, each request is served by a new service instance. It means a new service instance is created for each request and destroyed after the request is served.



This makes your WCF service stateless means you can't maintain states between WCF calls.

2. **Per Session** - In this way, each and every client requests are served by a new single service instance. It means a new service instance is created for a client all requests and destroyed after the client finished its activity.



This way makes your WCF service state-full means you can maintain states between WCF calls.

3. **Single** – In this way, all clients' requests are served by a single service instance. It means a single service instance is created to handle all the clients' requests and never destroyed after serving the request.



This makes your WCF service data shared globally.

Q4. How WCF Session is different from ASP.NET Session?

Ans. WCF Session management is different from ASP.NET Session Management in the following ways:

1. WCF Sessions are created and terminated by service client.
2. Server use session to manage service instances, not to store some general data like ASP.NET.
3. WCF Session does not mainly rely on Session ID like ASP.NET, a particular field within the header or body of a group of messages can also be considered as a part of the session.
4. There may be different Session ID at client and server side unlike ASP.NET

Q5. What is Concurrency Management in WCF?

Ans. Concurrency management is closely related to the Instance management in WCF but both are two different things. Instance management specifies how the service instances are created while Concurrency

management specifies how many concurrent requests are handled by the service instances. By using concurrency, you can make your service instance thread-safe.

Q6. Where Concurrency Management is required in WCF?

Ans. By default, a **per-call** service instance is thread-safe since each request is served by a new service instance. Hence, it's not required concurrency management.

A **per-session** service instance is not thread-safe since multiple requests of a client are served by a single service instance. Hence, it's required concurrency management.

A **single service** instance is not thread-safe since multiple requests of all clients are served by a single service instance. Hence, it's required concurrency management.

Q7. What are various Concurrency Modes in WCF?

Ans. WCF supports three ways of concurrency management which are defined by the *ConcurrencyMode* property of the *ServiceBehavior* attribute. These are as given below:

1. Single
2. Multiple
3. Reentrant

Q8. Do explain each Concurrency Modes in WCF?

Ans. Each concurrency mode is explained as given below:

1. **Single** - In this way, a single request would be handled by a service instance at a time means single thread for an instance. Hence other next requests need to wait until the previous request is served. All these requests are placed in a waiting queue and are served out of the queue in order.

This is the default concurrency mode in WCF.

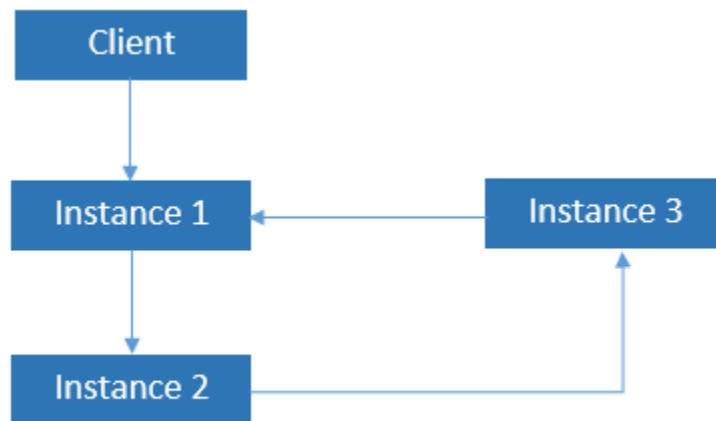
```
class MyService: IService
{
    public string GetData(int value)
    {
        throw new NotImplementedException();
    }
}

//Or
[ServiceBehavior(ConcurrencyMode = ConcurrencyMode.Single)]
class MyService : IService
{
    public string GetData(int value)
    {
        throw new NotImplementedException();
    }
}
```


2. **Multiple** - In this way, multiple requests would be handled by a service instance at a time means multiple threads for an instance to handle the requests.

```
[ServiceBehavior(ConcurrencyMode = ConcurrencyMode.Multiple)]
class MyService : IService
{
    public string GetData(int value)
    {
        throw new NotImplementedException();
    }
}
```

3. **Reentrant** - In this way, a single request would be handled by a single service instance at a time, but the request thread can call another service instance through callback and re-enter without deadlock. This is the refinement of Single Concurrency mode in WCF.



WCF: Reentrant

```
interface IServiceCallback
{
    [OperationContract]
    void OnCallback();
}
[ServiceContract(CallbackContract = typeof(IServiceCallback))]
interface IService
{
    [OperationContract]
    void MyOperation();
}
[ServiceBehavior(ConcurrencyMode = ConcurrencyMode.Reentrant)]
class MyService : IService
{
    public void MyOperation()
    {
        IServiceCallback callback = OperationContext.Current.
            GetCallbackChannel<IServiceCallback>();
        callback.OnCallback();
    }
}
```

Q9. How Instancing mode and Concurrency mode are related to each other?

Ans. There are nine combinations of Instancing mode and Concurrency mode as given below:

Instancing Mode	Concurrency Mode		
	Single (Default)	Multiple	Reentrant
Single	Single thread for handling all client's requests	Multiple threads for handling all client's requests	Single thread for handling all clients request and locks on data would be released when service call another service through a callback
Per-Session (Default)	Single thread for handling each client request.	Multiple threads for handling each client request	Single thread for handling all clients request and locks on data would be released when service call another service through a callback
Per-Call	Single thread for handling each client request	Single thread for handling each client request	Single thread for handling all clients request and locks on data would be released when service call another service through a callback

Q10. What is Throttling in WCF?

Ans. Throttling is a behaviour of WCF which allow you to define a **maximum** number of concurrent calls, concurrent active instances, and concurrent active sessions. These three (concurrent calls, concurrent active instances, and concurrent active sessions) can be defined within the serviceBehaviors tag of your service as –

```
<serviceBehaviors>
  <behavior name="MyServiceBehavior">
    <serviceThrottling maxConcurrentCalls="20"
      maxConcurrentInstances="50000000" maxConcurrentSessions="10" />
  </behavior>
</serviceBehaviors>
```

1. **MaxConcurrentCalls**- Define the maximum number of concurrent requests that can be handled by a WCF service instances.
2. **MaxConcurrentInstances**- Define the maximum number of active instances at a time. For the per-call session, it would be equal to the max no of concurrent calls. For per-session mode, it would be equal to the max no of active sessions. For single session mode, this setting doesn't matter since only one instance is ever created.
3. **MaxConcurrentSessions**- Define the maximum number of active sessions for a WCF service.

In WCF 4.0 all the above settings also depend upon the no of the processor in your machine. These would be the multiple of no of the processor in your machine.

Q11. What are message encoding and decoding in WCF?

Ans. Message encoding is a process of converting a message into a sequence of bytes and Message decoding is the reverse process of Message encoding.

Q12. What are different types of message encoding in WCF?

Ans. WCF supports three types of encoding for messages as given below:

1. **Text** – This encoding supports both plain XML encoding and SOAP encoding. It provides the widest interoperability compared to others since it is compatible with ASMX services and other services like java services.
2. **Binary** – This encoding produces the smallest message size among the others encoding. This suitable for WCF to WCF communication means both service and consumer should be WCF-based. This is the most efficient format.
3. **MTOM (Message Transmission Optimization Mechanism)** – This encoding is good for objects containing binary data in interoperable scenarios. But it produces plain XML for normal object properties. This also provides significant interoperability because W3C has standardized MTOM.



Transaction Management

Q1. What are different types of transaction in WCF?

Ans. Transaction in WCF ensures that service operations are executed successfully or failed as a unit. A successfully executed transaction moves the system from one consistent state to another consistent state. If the transaction encounters an error while its execution then it rolls back to the previous consistent state.

WCF supports two types of transactions as given below:

1. **Local Transaction**- Runs within the service domain boundary means within the same service.
2. **Distributed Transaction**- Runs across the service domain boundary means client flows its transaction to the service. A distributed transaction may also contain two or more independent services within the client transaction.

Q2. How you can configure Distributed Transaction in WCF?

Ans. By default, the distributed transaction is disabled in WCF. To enable distributed transaction first you need to configure WCF binding that supports the transaction and the second thing you need to configure *TransactionFlow* and *TransactionScopeRequired* attributes. The auto-commit behaviour of a WCF service is controlled by *TransactionAutoComplete* attribute and by default its value is true.

WCF distributed transactions require MSDTC (Microsoft Distributed Transaction Coordinator) running on all machines participating in the transaction i.e. the client, the WCF server and the database server.

Q3. Which protocol support Transaction in WCF?

Ans. WS-Atomic protocol is used to manage transactions in the WCF.

Q4. Which WCF Bindings support Transaction?

Ans. There are very few WCF bindings which support transactions. These are *WSHttpBinding*, *WSDualHttpBinding*, *NetTcpBinding*, *NetNamedPipeBinding* and *WSFederationHttpBinding*.

Q5. What is Transaction Flow or Transaction Propagation?

Ans. In WCF, a transaction can be propagated across the service boundary. This enables a WCF service to participate within the transaction at the client side. By default, WCF doesn't allow transaction propagation. For transaction propagation, you need to enable it in the binding on service host and client side.

Q6. What is various Transaction Flow?

Ans. There are three types of transaction flow which are set by **TransactionFlowAttribute** by using TransactionFlowOption as given below:

1. Not Allowed
2. Allowed
3. Mandatory

Q7. Explain all the Transaction Flow options?

Ans. All the transaction flow options are described as follows:

1. **NotAllowed** - This disallows the transaction propagation means the client cannot flow its transaction to the service. This is the default value for TransactionFlowOption property and the operation cannot participate in a transaction.

```
[ServiceContract]
interface IService
{
    [OperationContract]
    void MyOperation();
}
class MyService : IService
{
    [TransactionFlow(TransactionFlowOption.NotAllowed)]
    public void MyOperation()
    {
        //To DO:
    }
}
```

2. **Allowed** - This allows the transaction propagation means the client can flow its transaction to the service. The operation can participate in a transaction if the client creates one.

```
[ServiceContract]
interface IService
{
    [OperationContract]
    void MyOperation();
}
class MyService : IService
{
    [TransactionFlow(TransactionFlowOption.Allowed)]
    public void MyOperation()
    {
        //To DO:
    }
}
```

3. **Mandatory** - This implies that the client must have a transaction to call the service and it also allows the client to flow its transaction to the service.

```
[ServiceContract]
interface IService
{
    [OperationContract]
    void MyOperation();
}
class MyService : IService
{
    [TransactionFlow(TransactionFlowOption.Mandatory)]
    public void MyOperation()
    {
        //To DO:
    }
}
```

Q8. What is Transaction Scope?

Ans. A transaction scope defines a block of code that participates in a transaction. If the code block completes successfully, the transaction manager commits the transaction. Otherwise, the transaction manager rolls back the transaction.

Q9. How to allow or disallow a Transaction in WCF?

Ans. *TransactionScopeRequired* attribute is used to allow or disallow the transaction within the service method. By default, its value is false means it disallows the use of transaction with the service method. To allow a transaction within a service method you need to set *TransactionScopeRequired=true* as given below:

```
[ServiceContract]
interface IService
{
    [OperationContract]
    void MyOperation();
}
class MyService : IService
{
    [TransactionFlow(TransactionFlowOption.Allowed)]
    [OperationBehavior(TransactionScopeRequired = true)]
    public void MyOperation()
    {
        using (TransactionScope scope = new TransactionScope())
        {
            //To Do
            scope.Complete();
        }
    }
}
```

Q10. How WCF automatically completes the Transaction?

Ans. WCF automatically completes the active transaction by using *TransactionAutoComplete* attribute. By default, its value is true means WCF automatically completes the active transaction. When you set this attribute value to false, you need to complete the transaction explicitly.

Automatically Completed Transaction:

```
[ServiceContract]
interface IService
{
    [OperationContract]
    void MyOperation();
}
class MyService : IService
{
    [TransactionFlow(TransactionFlowOption.Allowed)]
    //Use of TransactionAutoComplete is optional
    [OperationBehavior(TransactionScopeRequired = true, TransactionAutoComplete = true)]
    public void MyOperation()
    {
        using (TransactionScope scope = new TransactionScope())
        {
            //Do some work then
            scope.Complete();
        }
    }
}
```

Explicitly Completed Transaction:

```
[ServiceContract]
interface IService
{
    [OperationContract]
    void MyOperation();
}
class MyService : IService
{
    [TransactionFlow(TransactionFlowOption.Allowed)]
    [OperationBehavior(TransactionScopeRequired = true, TransactionAutoComplete = false)]
    public void MyOperation()
    {
        using (TransactionScope scope = new TransactionScope())
        {
            //Do some work then
            scope.Complete();
        }
        //Explicitly completes the transaction
        OperationContext.Current.SetTransactionComplete();
    }
}
```

Q11. Can we implement transactions in one-way WCF service calls?

Ans. No, you cannot. Since in one-way communication, you will not get the response back. But for the transaction implementation, the returned response is necessary.

6

Security and Versioning

Q1. What is Impersonation?

Ans. Impersonation is a way to authorize a caller (client) identity to access the service domain's resources. The service resources may be local files or database tables and should be on the same machine on which service is hosted. By default, impersonation is disabled and resources are accessed by using the WCF service's process identity.

Q2. What are the different types of Impersonation?

Ans. There are two types of impersonation techniques as given below:

1. **Imperative Impersonation** – This impersonation is performed programmatically at runtime and can vary depending on business logic or other conditions.

```
[ServiceContract]
public interface IService
{
    [OperationContract]
    string Greet(string name);
}

public class MyService: IService
{
    [OperationBehavior]
    public string Greet(string name)
    {
        WindowsIdentity callerWindowsIdentity =
        ServiceSecurityContext.Current.WindowsIdentity;
        if (callerWindowsIdentity == null)
        {
            throw new InvalidOperationException
            ("The caller cannot be mapped to a windows identity");
        }
        using (callerWindowsIdentity.Impersonate())
        {
            // Access a file as the caller.
        }
        return "Hello, " + name;
    }
}
```


2. **Declarative Impersonation** – This impersonation is applied with a static attribute that can be associated with an operation or an entire interface.

```
[ServiceContract]
public interface IService
{
    [OperationContract]
    string Greet(string name);
}

public class MyService: IService
{
    [OperationBehavior(Impersonation = ImpersonationOption.Required)]
    public string Greet(string name)
    {
        return "Hello, " + name;
    }
}
```

Q3. What are Impersonation Levels?

Ans. Impersonation levels allow the server process to act on behalf of a client process.

Q4. What are various Impersonation Levels?

Ans. The various impersonation levels are as -

1. **Anonymous**- In this, the server process cannot impersonate the client since it doesn't receive the user's identification information.
2. **Delegation**- In this, the server process can impersonate the client's security context on the remote machine.
3. **Identification**- In this, the server process can get information about the client identification, but it cannot impersonate the client.
4. **Impersonation**- In this, the server process can impersonate the client's security context on its local machine, but it cannot impersonate the client on remote machines.
5. **None**- An impersonation level is not assigned.

Q5. What is Delegation?

Ans. Delegation is a technique that allows WCF service to access the network resources on behalf of the original caller. This technique delegates the client's request from the front-end service to the back-end service in such a way that the back-end service can also impersonate the client.

Q6. How to configure Delegation?

Ans. To delegate the client's request from the front-end service to the back-end service, you need to create a *ChannelFactory* and a channel. Using channel, you can communicate to the backend service and also impersonate the client. To enable delegation-level impersonation you need to configure Windows domain policy to permit delegation.

```

[ServiceContract]
public interface IService
{
    [OperationContract]
    string Greet(string message);
}

public class MyService: IService
{
    [OperationBehavior(Impersonation = ImpersonationOption.Required)]
    public string Greet(string message)
    {
        WindowsIdentity callerWinIdentity =
        ServiceSecurityContext.Current.WindowsIdentity;
        if (callerWinIdentity == null)
        {
            throw new InvalidOperationException
            ("The caller cannot be mapped to a Windows identity.");
        }
        using (callerWinIdentity.Impersonate())
        {
            EndpointAddress backendServiceAddress = new
            EndpointAddress("http://localhost:8080/ChannelService");

            // Calling Backend Service
            ChannelFactory<IService> channelFactory = new ChannelFactory<IService>(new
            NetTcpBinding(), backendServiceAddress);
            IService channel = channelFactory.CreateChannel();
            return channel.Greet(message);
        }
    }
}

```

Q7. What are various delegation options in WCF?

Ans. There are following delegation options in WCF:

1. **Use Kerberos authentication and delegation** – When you are using Kerberos to authenticate users, you can impersonate the original caller by using below techniques –
 - **WCF with Network Service account** – In this case, you need to configure your computer account in Active Directory to be trusted for delegation.
 - **WCF with custom domain account** – In this case, you need to register a service principal name (SPN) in Active Directory to associate the domain account with the HTTP service on your WCF server. You then configure your domain account in Active Directory to be trusted for delegation.
2. **Non-Kerberos authentication (Protocol transition)** - You can use client certificates to authenticate users and then use the new *WindowsIdentity* constructor to obtain windows token for the user on the server.
3. **Interactive Logon Session** - An interactive login session has network credentials that allow the user to authenticate against network servers. Use this approach when you cannot use the above two approaches.

Q8. What are the differences between impersonation and delegation?

Ans. The differences between Impersonation and Delegation are listed below –

1. Impersonation is a way to authorize a caller (client) identity to do a particular action, while delegation is a way of flowing impersonation capabilities along with the caller identity to a back-end service.
2. Impersonation is a way to access the resources which are on the same machine where the service is hosted. Delegation is a way to access the resources on the remote machine.

Q9. What are WCF security features?

OR

What are the main principles in web service security?

Ans. Security is the main concern for web services at the enterprise level. There are four main principles for web service security which is listed below-

1. **Authentication** - This is the process of verifying the caller's identity. WCF provides several options for authentications like as windows, basic, certificate and issued token.
2. **Authorization** - This is the process of identifying the authenticated caller's access right for the service operations and system resources.
3. **Confidentiality** - This process ensures that the message passed from the sender to the receiver remains private and confidential. It cannot be read/access by unauthorized users.
4. **Integrity** - This process ensures that the message passed from the sender to the receiver remains protected from accidental or deliberate modifications. The receiver will get the same message as it is passed from the sender.

Q10. What is infrastructure level security in WCF?

OR

What are various levels of security provided by WCF?

Ans. WCF supports three ways for implementing security at the infrastructure level which is given below-

1. **Transport Security** – Transport level (i.e., protocol level) security provides a secure end to end communication between the client and the service. This security happens at the channel level. WCF supports various transfer protocols like TCP, HTTP and MSMQ etc. and each protocol has its own security implementation. The most common approach is to use SSL (Secure Socket Layer) to protect data over HTTP protocol.



The goal of transport level security is to provide authentication, confidentiality and integrity.

2. **Message Security** – In message level security, the message is encrypted at client side before sending it to the service and same message is decrypted at service side and vice versa by using some standard encryption algorithms. This security does not depend on the protocols supported by WCF. This is based on the WS- Security specification which specify how to encrypt message's data or SOAP envelope by using different encryption algorithms.



Transport level security is faster than message-level security since message level security is based on XML serialization and deserialization, which makes it slower than transport level security.

3. **Mix Mode or TransportWithMessageCredential Security** – This is the combination of the above two security – Transport level and Message level. Mixed mode security provides integrity and confidentiality at the transport level, and authentication and authorization at the message level. It also offers a secure and fast way of transmitting data between services and clients.

Q11. What are the various security modes in WCF?

Ans. WCF provides security on the transport level and message level. Based on bindings, WCF offers various security mode as given below –

1. **None** – This mode doesn't provide any security and all the information is passed in clear text. By default, BasicHttpBinding uses this mode, but the other bindings do not use it.
2. **Transport** – This mode uses transport level security and provides mutual authentication and message protection at the transport level.
3. **Message** – This mode uses message level security and provides mutual authentication and message protection at the message level.
4. **Both** – This mode is only available in the *NetMsmqBinding* binding and provides mutual authentication and message protection at both the transport and message level.

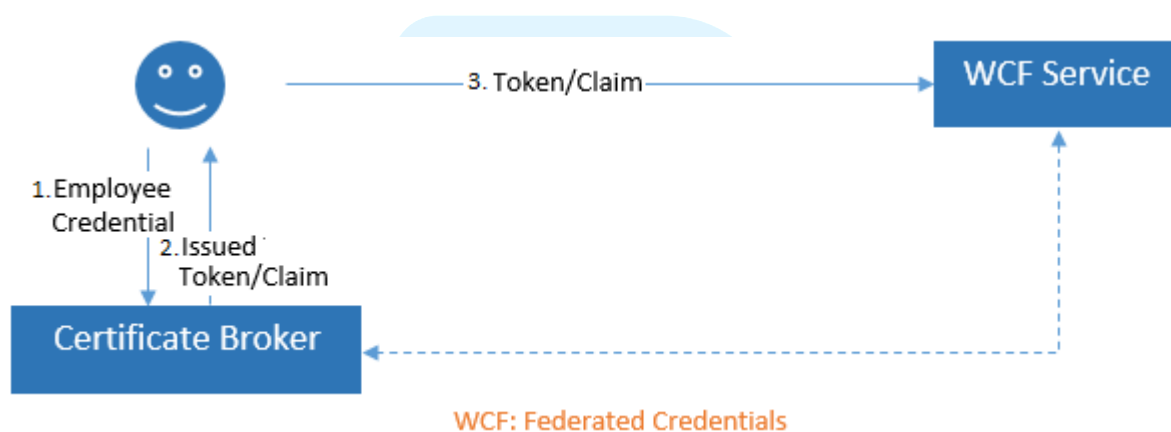
5. **TransportWithMessageCredential** - This mode provides client authentication at the message level and message protection and service authentication at the transport level.
6. **TransportCredentialOnly** – This mode uses transport level security and provides mutual authentication at the transport level. This doesn't provide any message protection. This mode is available only in the *basicHttpBinding*.

Q12. What is Federated Security in WCF?

Or

What is Brokered Security in WCF?

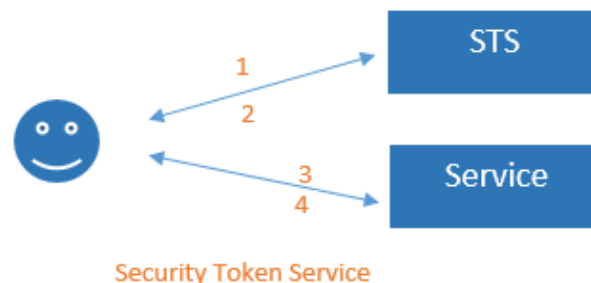
Ans. Today, federation security is a popular model for distributed services. In this security, verification of a claim (i.e. user) is done by a third-party vendor/broker known as security token providers. In response to a user, this third-party vendor provides a token key (certified credential) through which user can communicate with the service.



This third-party vendor has a trust relationship with the service. Therefore, whenever a user comes with a certified credential provided by the service vendor, the service will trust the client's credentials.

Q13. What is a Security Token Service (STS)?

Ans. Security Token Service is provided by the third-party vendor which has a trust relationship with the service. To call the service, the user requests a security token from the third-party vendor. The third-party vendor issued a security token to the user. By using this token user can access the service.



Q14. What is the difference between Credential and Claim?

Ans. A credential is an XML compatible entity that helps the Windows runtime in identifying a user. A claim is a part of credential and a credential can have more than one claims. A claim can be a username, digital certificate or token that specifies the user identity.

Q15. What is the protection level in WCF?

Ans. A Protection level ensures how a part or whole of a message is protected. You can apply for protection at the following levels:

- **Service level** to affect all parts of every message
- **Operation level** to affect a particular message.
- **Message-level** to affect all parts of a particular message.
- **Message parts level** to affect a part like header, body etc. of a particular message.

Q16. How to set protection level in WCF?

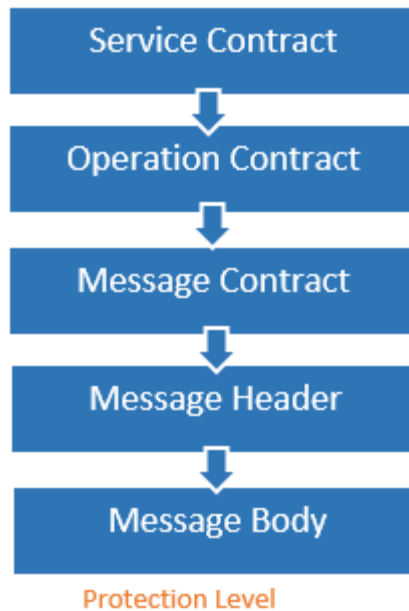
Ans. You can set the protection at a various level as given below:

Service Contract Level

```
[ServiceContract(ProtectionLevel = ProtectionLevel.Sign)]
interface IService
{
    [OperationContract]
    void MyOperation();
}
```

Operation Contract Level

```
[ServiceContract]
interface IService
{
    [OperationContract(ProtectionLevel = ProtectionLevel.Sign)]
    void MyOperation();
}
```



Message Contract Level

```
[MessageContract(ProtectionLevel = ProtectionLevel.EncryptAndSign)]
public class Client
{
    [MessageHeader(ProtectionLevel = ProtectionLevel.Sign)]
    public string Name;

    [MessageBodyMember(ProtectionLevel = ProtectionLevel.EncryptAndSign)]
    public string ID;
}
```

This can be set only by code, not in configuration.

Q17. What are different types of protection level?

Ans. There are three types of protection level in WCF as given below –

1. **None** – This does not protect the message on transfer from the sender to the receiver means anyone can read the message.
2. **Signed** – This ensures the message protection and message will be signed digitally before sending to the receiver. This level could not hide the message content during the transfer.
3. **Encrypted and Signed** - This ensures the message protection and message will be encrypted and signed digitally before sending to the receiver. This level hides the message content during the transfer. Hence this level provides integrity, privacy, and authenticity.

Q18. What is Authentication? What are authentication options in transport level security?

Ans. Authentication is the process of verifying the user identity. Only a verified user can call the WCF service.

WCF transport level security offers the following authentication ways –

1. **None**– This option does not authenticate the service’s clients and allow all callers to call the service. This is used for anonymous users.
2. **Basic** – This option is available with the HTTP protocol only and service’s clients are authenticated by using username and password against active directory. The client credentials are passed using the Base64 encoded the string, which is like a clear string. The service is authenticated by the SSL certificate which is used for secure communication.
3. **NTLM** - This option is available with the HTTP protocol only and service’s clients are authenticated using a challenge-response scheme against Windows accounts. This option is well suited for a workgroup environment and more secure than Digest or Basic authentication. The service is authenticated by the SSL certificate or Windows credentials of the process identity.

Binding	None	Windows	Username	Certificate
BasicHttpBinding	Yes (default)	Yes	Yes	Yes
WSHttpBinding	Yes	Yes (default)	Yes	Yes
NetTcpBinding	Yes	Yes (default)	No	Yes
NetNamedPipeBinding	No	Yes (default)	No	No
NetMsmqBinding	Yes	Yes (default)	No	Yes

4. **Windows** – This option uses Kerberos authentication when in a domain, or NTLM when deployed in a workgroup environment. This option uses a Windows token presented by the client to authenticate against Active Directory and more secure than Basic, Digest, or NTLM authentication. The service is authenticated by Windows credentials of the process identity or an SSL certificate if you are using the HTTP protocol.
5. **Certificate** – This option uses *X509 certificate* to authenticate the client. Typically, the client’s certificate is known in advance to the service. Hence the service looks up the certificate on the host side and authenticates the client.

Q19. What is the authentication option in Message-Level Security?

Ans. WCF Message-Level Security offers the following authentication ways –

1. **None**– This option does not authenticate the service’s clients and allow all callers to call the service. This is used for anonymous users.

2. **Username** – This option is available with the HTTP protocol only and service's clients are authenticated by using username and password against Active Directory. The client credentials are passed using the Base64 encoded the string, which is like a clear string. The service is authenticated by the SSL certificate which is used for secure communication.
3. **Windows** – This option uses Kerberos authentication when in a domain, or NTLM when deployed in a workgroup environment. This option uses a Windows token presented by the client to authenticate against Active Directory and more secure than Basic, Digest, or NTLM authentication. The service is authenticated by Windows credentials of the process identity or an SSL certificate if you are using the HTTP protocol.

Binding	None	Windows	Username	Certificate	Issued Token
BasicHttpBinding	No	No	No	Yes	No
WSHttpBinding	Yes	Yes(default)	Yes	Yes	Yes
NetTcpBinding	Yes	Yes(default)	Yes	Yes	Yes
NetNamedPipeBinding	N/A	N/A	N/A	N/A	N/A
NetMsmqBinding	Yes	Yes(default)	Yes	Yes	Yes

4. **Certificate** – This option uses *X509 certificate* to authenticate the client. Typically, the client's certificate is known in advance to the service. Hence the service looks up the certificate on the host side and authenticates the client.
5. **Issue token**- This option uses an issued token by STS (secure token service) provider to authenticate the client.

Q20. What is authorization? What is the various authorization approach?

Ans. Authorization is the process of identifying the authenticated caller's access right for the service operations and system resources. WCF supports the following three basic authorization approaches:

1. **Role-based**- This authorization to access the WCF operations is based on the role membership of the client. The role store can be Windows groups, ASP.NET roles (SqlRoleProvider, WindowsTokenRoleProvider), or a custom role store.
2. **Identity-based**- This authorization to access the WCF operations is based on Identity Model which is an extension to role-based authorization. This model allows you to manage claims and policies to authorize clients. Hence by using this model, you can authenticate the clients.
3. **Resource-based**- This authorization to access the Individual resources is based on Windows access control lists (ACLs). The WCF service impersonates the client before accessing the resources, which allows the operating system to perform standard access checks. All resource access is performed using the original caller's security context.

Q21. What is Service Auditing?

Ans. A service auditing is a way to log authentication or authorization attempts and other information such as their times, locations, and client identity. WCF service auditing allows you to detect an attack that has occurred or is in progress within the service. It also helps you to debug security-related problems.

WCF auditing can be enabled via configuration by using the `ServiceSecurityAuditBehavior` element. The event log location for auditing the events can be specified in the `auditLogLocation` attribute.

```
<system.serviceModel>
  <services>
    <service name = "MyService" behaviorConfiguration = "MySecurityAudit">
      <!--To Do :-->
    </service>
  </services>
  <behaviors>
    <serviceBehaviors>
      <behavior name = "MySecurityAudit">
        <serviceSecurityAudit auditLogLocation="Default" serviceAuthorizationAuditLevel
="SuccessOrFailure" messageAuthenticationAuditLevel="SuccessOrFailure"/>
      </behavior>
    </serviceBehaviors>
  </behaviors>
</system.serviceModel>
```

The possible values for audit location are `Default`, `Application` and `Security`. When you select `Default`, the operating system decides the actual log location. The possible values for `serviceAuthorizationAuditLevel` and `messageAuthenticationAuditLevel` are `None`, `Failure`, `Success`, and `SuccessOrFailure`.

Q22. What is service versioning?

Ans. After initial deployment of the WCF service, you may need to change the service for a variety of reasons like as changing business needs or fixing others issues. Each change in your existing service introduces a new version of the service. Service versioning is helpful in backward compatibility with your existing clients.

Q23. What is the different level of versioning?

Ans. Typically, WCF service changes at service contract level or at the data contract level. Hence you need to implement versioning at two levels as given below –

1. **Service Contract Versioning** – In this versioning new operations can be added to the existing service but the existing operations cannot be removed or changed.
2. **Data Contract Versioning** - In this versioning new schema type definitions or new members can be added but existing schema type definitions or members cannot be changed. This feature allows the change in the version of data contracts without breaking clients.

Q24. What are various version strategies in WCF?

Ans. There are some versioning strategies as given below -

1. **Strict Versioning** - In this versioning, a service developer didn't change the existing data contracts and create new ones with a new signature. In this way, a service developer either add new operations to an existing service contract or create a new service contract with operations that use the new data contract.
2. **Lax Versioning** - In this versioning, a service developer makes the assumption that adding a new or optional member to the data contract will not break existing clients. This will work if the existing clients are not performing schema validation or they ignore unknown data members.

WCF, ASP.NET Web Services, and many other Web service stacks support lax versioning and they do not throw exceptions for new unknown data members in received data.



WCF REST

Q1. What is REST?

Ans. REST stands for **Representational State Transfer**. This is a protocol for exchanging data over a distributed environment. REST is an architectural style which treats each service as a resource and access data by HTTP protocol methods like GET, POST, PUT, and DELETE.

REST-style architectures consist of clients and servers. Clients initiate requests to servers who process these requests and return responses based on these requests. These requests and responses are built around the transfer of representations of these resources.

Q2. Explain the REST principle?

Ans. REST is a set of principles that define how Web standards such as HTTP and URIs, are supposed to be used. There are five important REST principle as given below –

- **Addressable Resources**- Each resource should be identified by a URI (unique identifier)
- **Simple and Uniform Interfaces** – REST is based on HTTP protocol so use HTTP GET, POST, PUT and DELETE method to perform actions. This makes REST simple and uniform.
- **Representation Oriented** - Representation of resources are exchanged. GET is used to return a representation and PUT, POST passes representation to the server so that underlying resources may change. Representation may be in many formats like XML, JSON etc.
- **Communicate Stateless** - The application may have state but there is no client session data stored on the server. Any session specific data it should be held and maintained by the client and transferred to the server with each request as needed.
- **Cacheable** - Clients should be able to cache the responses for further use.

Q3. What is ROA?

Ans. ROA is a service architecture based on the REST style. It is based upon the concept of a resource. A resource is any information that has a name (like image, document etc.) and an address represented by a URI. ROA is also considered a RESTful architecture.

Q4. What binding is used for WCF REST?

Ans. The *WebHttpBinding* is used for WCF REST service.

Q5. What protocols can be used with WCF REST?

Ans. A WCF REST service is accessed over only HTTP protocol verbs (GET, POST, PUT, DELETE).

Q6. What is the difference between REST and SOAP?

Ans. The difference between REST and SOAP is given below:

SOAP	REST
SOAP stands for Simple Object Access Protocol	REST stands for REpresentational State Transfer.
It is an XML based protocol built on the top of HTTP or sometimes TCP/IP, SMTP.	REST is not a protocol but it is an architectural style i.e. resource-based architecture.
SOAP has specifications for both stateless and stateful implementation.	REST is completely stateless.
SOAP enforces message format as XML.	REST does not enforce message format as XML or JSON.
SOAP has a defined standard specification. For example, WS-Security is the specification for implementing security.	It has no defined standard specifications.
The SOAP message consists of an envelope which includes SOAP headers and body to store the actual information you want to send.	REST uses the HTTP protocols features such build-in headers and use the GET, POST, PUT and DELETE verbs to perform operations.
SOAP uses interfaces and named operations to expose your service.	REST uses URI and methods like (GET, PUT, POST, DELETE) to expose resources.
Performance is slow as compared to REST.	REST is fast as compared to SOAP.

Q7. Explain HTTP GET, POST, PUT and DELETE methods?

Ans. The HTTP protocol methods are as:

1. **GET** - This method is used to retrieve the data from the remote data source. This represents **R** (retrieve) of CRUD operation.
2. **POST** - This method is used to update the current record of the data on the remote data source. This represents **U** (Update) of CRUD operation. POST can also be used to creating and delete.
3. **PUT** - This method is used to create a new record for the current data that is being sent to the remote data source. This represents **C** (Create) of CRUD operation.
4. **DELETE** - This method is used to delete the specified data from the remote data source based upon id. This represents **D** (Delete) of CRUD operation.

Q8. What is UriTemplate?

Ans. UriTemplate is a property of *WebGet* and *WebInvoke* attribute which helps us to map the parameter names coming from the HTTP request with the parameter names of WCF operation contract as given below -

```
[ServiceContract]
public interface IProductService
{
    [OperationContract]
    [WebInvoke(UriTemplate = "AddProduct/{name}")]
    //mapped to http://localhost/ProductService/AddProduct/CSharp
}
```

```

void AddProduct(string name);

[OperationContract]
[WebInvoke(UriTemplate = "UpdateProduct/{id}/{name}")]
//mapped to http://localhost/ProductService/UpdateProduct/1/VB.NET
void UpdateProduct(string id, string name);

[OperationContract]
[WebInvoke(UriTemplate = "DeleteProduct/{id}")]
//mapped to http://localhost/ProductService/DeleteProduct/1
void DeleteProduct(string id);
}

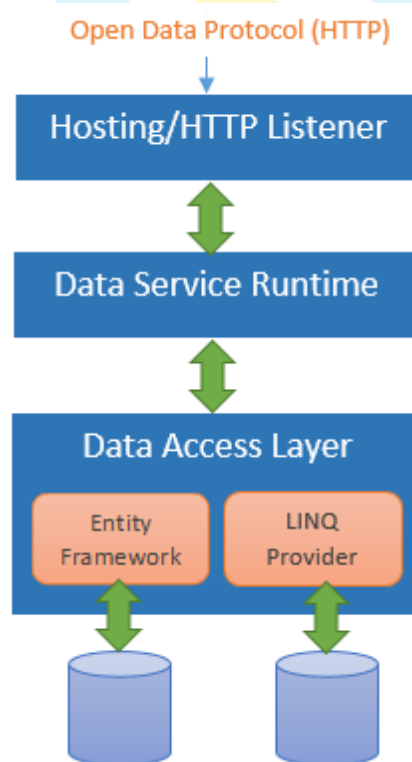
```

Q9. What is WCF RIA service?

Ans. A WCF RIA Service is designed to work with Rich Internet Applications (RIA) such as Silverlight applications that need a clearly separated middle tier. Now, WCF RIA also supports OData, SOAP, and JSON.

Q10. What is WCF Data Service?

Ans. WCF Data Services uses OData (Open Data Protocol) protocol for querying or manipulating the data. WCF Data Services is built on top of WCF REST Services. It is a RESTful service to support CRUD operations on the database using the HTTP protocol. It supports all database operations using URI. ODATA protocol can expose data from the relational database, File systems, Web sites, services etc. It supports XML or JSON format for exposing the data.



For creating WCF Data service, you need to include WCF Data Service template into your visual studio.

References

This book has been written by referring to the following sites:

1. <https://docs.microsoft.com/en-us/dotnet/framework/wcf/index> - Microsoft Docs - WCF
2. <https://stackoverflow.com/questions/tagged/wcf> - Stack Overflow - WCF
3. <https://www.dotnettricks.com/learn/wcf> - Dot Net Tricks - WCF

